

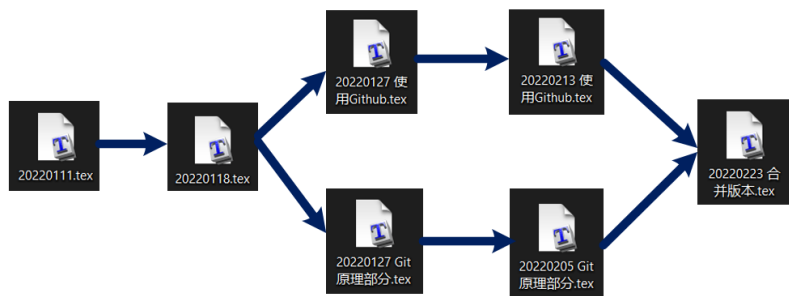
Git 笔记

Chapter1 Git 常识

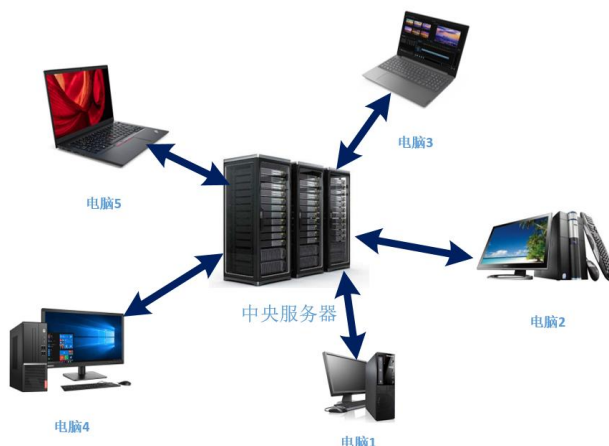
1. **版本控制**：进行多次开发，会有多个版本文件。可以将每一次的文件保存下来，但这种方法存在缺点。因此需要进行版本控制。

```
毕业论文_初稿.doc  
毕业论文_修改1.doc  
毕业论文_修改2.doc  
毕业论文_修改3.doc  
毕业论文_完整版1.doc  
毕业论文_完整版2.doc  
毕业论文_完整版3.doc  
毕业论文_最终版1.doc  
毕业论文_最终版2.doc  
毕业论文_死也不改版.doc  
...
```

- 多个文件，保留所有版本时，需要为每个版本保存一个文件。
- 协同操作，多人协同操作时，需要将文件打包发来发去。
- 容易丢失，被删除意味着永远失去... (可以选择网盘)



2. **版本控制软件**：在编写程序时，往往需要多次开发，并且保留一些历史版本，以及对不同的需求扩展到新的版本，**版本控制软件可以自动记录一些文件的增删改查，并且可以随时回溯到历史版本**。版本控制软件分为集中式版本控制和分布式版本控制。
 - a) **集中式版本控制**：如 SVN 和 CVS。**有一个单一的服务器来管理，这个服务器保存了所有文件的修订和版本**，开发人员连接到服务器上以后，就可以取出文件以及提交更新。我们想要更改和修订时，就需要给联网到服务器。必须联网。



- b) **分布式版本控制 distributed revision control**：一种版本控制的方式，它允许软件

开发者可以共同参与一个软件开发过程，但是不必在相同的网络系统下工作（可以分开单独进行，独立开发出新的版本）。**理论上并没有中央服务器，如果使用客户端提取文件则需要把整个代码仓库镜像完整拷贝下来，每个人都保存一个完整的版本库。**如果某一个服务器出现了故障，就可以使用任何一个本地仓库来恢复。在实际操作时，Git 也会拟定一个中央仓库，而当中央仓库出现错误时，则开发者可以新建一个新的中央仓库，然后将本地仓库同步到新的中央仓库上。



3. **Git**: 一个用 C 语言开发的分布式版本控制软件，它会自动帮你记录文件的改动（不需要自己手动把每一行改动都记录下来，我们可以自己写一个简单的说明），可以与同事一起协作编辑。

比喻/解释

存档 提交

Commit

时间线 历史

History

增删改文件 更改

Change

Git词汇

HEAD

替换临时测试用文件

9月7日11:18

删除 Assets/Resources/NPC/TestSoldier.png

删除 Assets/Resources/NPC/TestSoldier.png.meta

新增 Assets/Resources/NPC/Soldier.png

新增 Assets/Resources/NPC/Soldier.png.meta

修改 Assets/Prefabs/Soldier.Prefab

调整主角动画状态机

9月7日11:18

修改 Assets/Resources/Character/MainCharacter.controller

新增 Assets/Resources/Character/RunToidle.anim

修复跳跃状态未正确重置

9月6日17:07

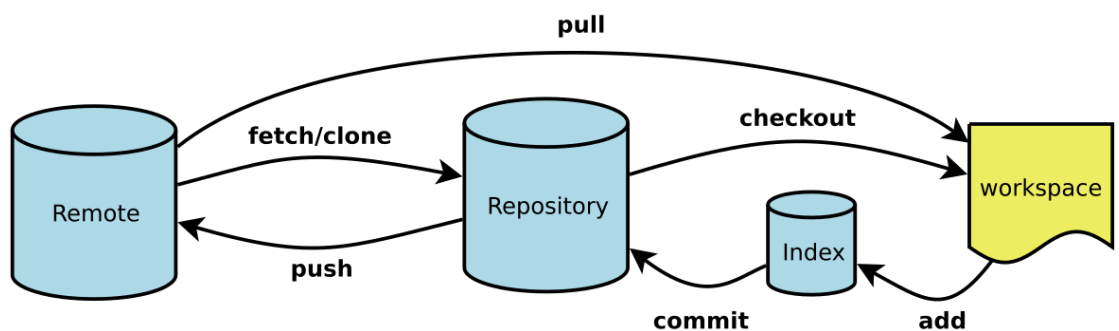
修改 Assets/Scripts/PlayerControl.cs

新增人物移动逻辑

9月6日15:35

新增 Assets/Scripts/PlayerControl.cs

每个提交记录的是被更改的文件，而非工作目录的所有文件



Workspace: 工作区，就是你平时存放项目代码的地方

Index / Stage: 暂存区，用于临时存放你的改动，事实上它只是一个文件，保存即将提交到文件列表信息

Repository: 仓库区（或版本库），就是安全存放数据的位置，这里面有你提交到所有版本的数据。其中HEAD指向最新放入仓库的版本

Remote: 远程仓库，托管代码的服务器，可以简单的认为是你项目组中的一台电脑用于远程数据交换

4. **代码托管平台：**代码托管平台是一种在线服务，用于存储、管理和共享软件开发项目的代码。这些平台通常提供了一个基于版本控制系统（如 Git）的仓库，开发人员可以将他们的代码提交到这些仓库中，并与团队成员协作。基于 Git 的托管平台主要有 Github 和 GitLab。

5. git 仓库配置设置

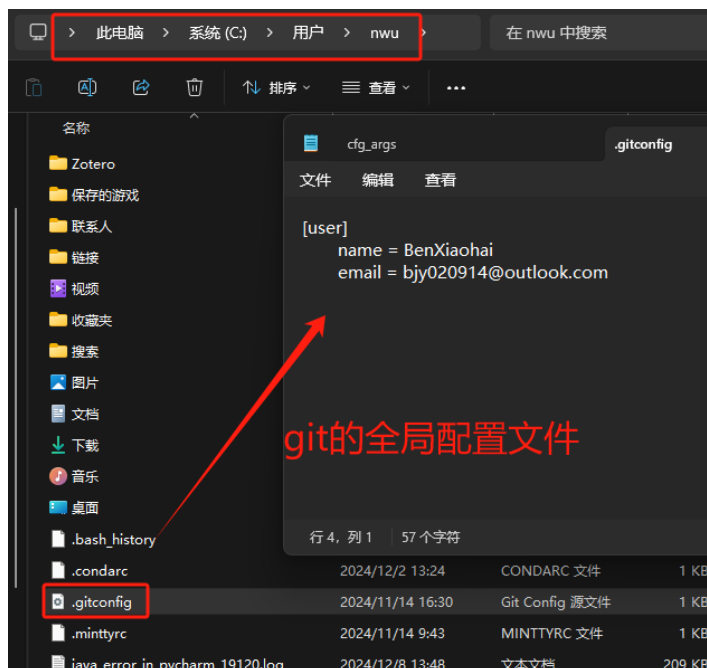
- a) **查看 git 仓库的配置文件：**可以查看当前仓库的配置文件(.git/config)，也可以查看全局的配置文件(当前用户 user/.gitconfig 文件)。

```
git-demo git:(master) cat .git/config
[core]
  repositoryformatversion = 0
  filemode = true
  bare = false
  logallrefupdates = true
  ignorecase = true
  precomposeunicode = true

git-demo git:(master) cat ~/.gitconfig
[filter "lfs"]
  clean = git-lfs clean %f
  smudge = git-lfs smudge %f
  required = true
[user]
  name = Peng Xiao
  email = xiaoquwl@gmail.com
[core]
  editor = vim
```

当前git仓库下的 .git文件夹下的 config文件

全局配置文件



- b) **配置用户：**Git 是分布式版本控制系统，每个用户都要进行备注。**--global 参数**表示全局配置，机器上所有的 Git 仓库都会设置为当前状态。不同的 Git 仓库也可以设置不同的用户名和 Email(**不添加—global 参数，分别设置**)。设置了用户名和邮箱以后，当把本地代码提交到远程仓库中时，远程仓库就会记录是谁提交的。

```
bjy02@BenXiaoHai MINGW64 ~
$ git config user.name

bjy02@BenXiaoHai MINGW64 ~
$ git config user.email

bjy02@BenXiaoHai MINGW64 ~
$ git config --global user.name "BenXiaoHai"

bjy02@BenXiaoHai MINGW64 ~
$ git config --global user.email "1987884531@qq.com"

bjy02@BenXiaoHai MINGW64 ~
$ git config user.name
BenXiaoHai

bjy02@BenXiaoHai MINGW64 ~
$ git config user.email
1987884531@qq.com
```

对机器上所有的 Git 仓库(global参数)都会设置为由当前用户管理

查看设置的用户

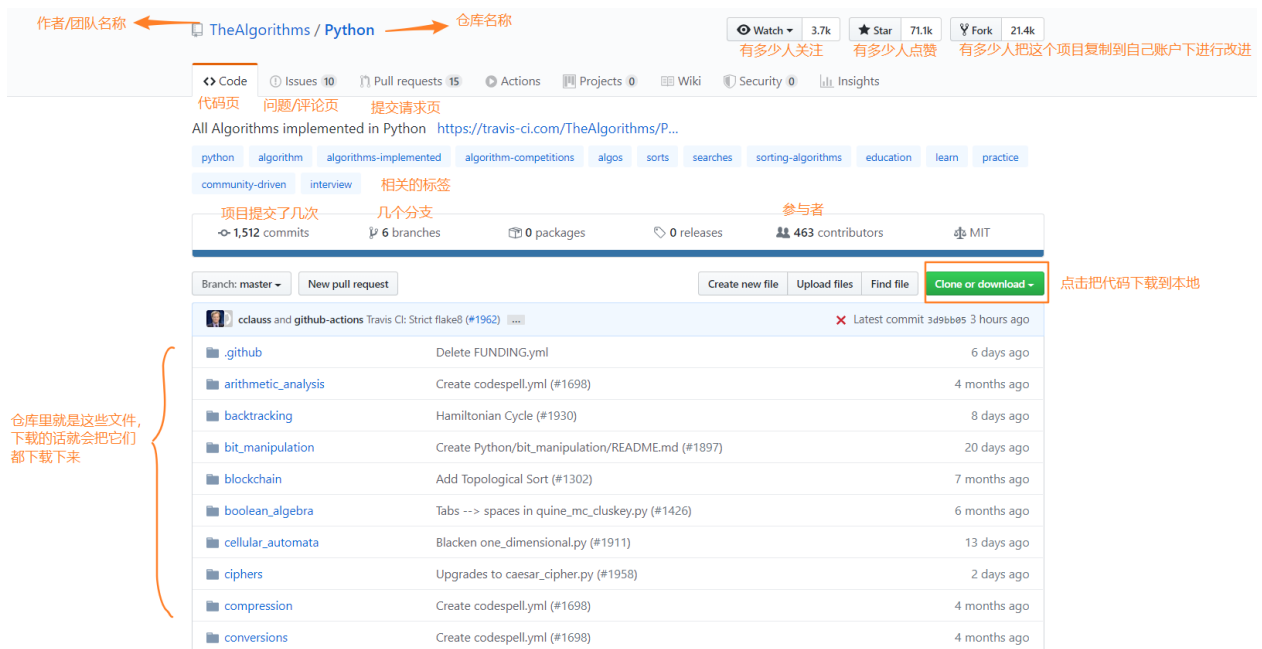
```
→ git-demo git:(master) git config user.name "demo"
→ git-demo git:(master) git config user.email "demo@demo.com"
→ git-demo git:(master) 配置时没有使用global参数, 默认进行本地仓库配置
→ git-demo git:(master) cat .git/config
[core]
    repositoryformatversion = 0
    filemode = true
    bare = false
    logallrefupdates = true
    ignorecase = true
    precomposeunicode = true
[user]
    name = demo
    email = demo@demo.com
→ git-demo git:(master) git config -l
→ git-demo git:(master) git config --global -l
→ git-demo git:(master)
→ git-demo git:(master) cat ~/.gitconfig
[filter "lfs"]
    clean = git-lfs clean %f
    smudge = git-lfs smudge %f
    required = true
[user]
    name = Peng Xiao
    email = xiaoquwl@gmail.com
[core]
    editor = vim
→ git-demo git:(master)
```

本地配置

全局配置

Chapter2 Git 的本地基础使用方法

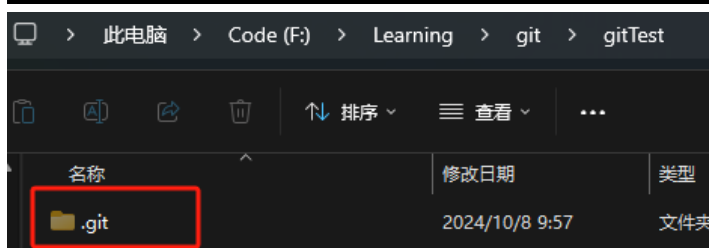
1. 版本库 repository: 也叫做代码仓库, 工作区有一个隐藏目录 `.git`, 称为 **Git 的版本库**。这个仓库里记录了所有文件的创建、更改和删除, 并记录了全部的历史, 以及可以进行还原。版本库可以是远程的, 也可以是本地的。



2. **git init**: 在指定要创建版本库的文件夹路径下，使用该命令创建一个 Git 管理的仓库。执行命令后，**自动生成了一个.git 目录（隐藏目录）**。该目录是用来管理和跟踪记录版本的，不要删除或任意修改。

```
→ github
→ github mkdir git-demo
→ github
→ github cd git-demo
→ git-demo
→ git-demo
→ git-demo ls
→ git-demo ls -la
total 0
drwxr-xr-x 2 pengxiao staff 64 May 15 00:22 .
drwxr-xr-x 9 pengxiao staff 288 May 15 00:22 ..
→ git-demo git init
Initialized empty Git repository in /Users/pengxiao/tmp/github/git-demo/.git/
→ git-demo git:(master) ls -la
total 0
drwxr-xr-x 3 pengxiao staff 96 May 15 00:26 .
drwxr-xr-x 9 pengxiao staff 288 May 15 00:22 ..
drwxr-xr-x 10 pengxiao staff 320 May 15 00:26 .git
→ git-demo git:(master)
```

```
./
./.git
./.git/config
./.git/objects
./.git/objects/pack
./.git/objects/info
./.git/HEAD
./.git/info
./.git/info/exclude
./.git/description
./.git/hooks
./.git/hooks/commit-msg.sample
./.git/hooks/pre-rebase.sample
./.git/hooks/pre-commit.sample
./.git/hooks/applypatch-msg.sample
./.git/hooks/fsmonitor-watchman.sample
./.git/hooks/pre-receive.sample
./.git/hooks/prepare-commit-msg.sample
./.git/hooks/post-update.sample
./.git/hooks/pre-applypatch.sample
./.git/hooks/pre-push.sample
./.git/hooks/update.sample
./.git/refs
./.git/refs/heads
./.git/refs/tags
./.git/branches
```



.git 目录下的 config 文件记录了本地版本库的本地配置文件

3. **git cat-file** 生成的文件哈希值 XXX: 查看 **objects** 文件某些属性。文件名通过哈希函数生成。

```

→ git-demo git:(master) x git cat-file -t 8d0e41
blob git cat-file: 查看objects文件          -t: type, 查看objects文件类型
→ git-demo git:(master) x git cat-file -p 8d0e41
hello git                                  -p: 查看objects文件内容
→ git-demo git:(master) x
→ git-demo git:(master) x cat hello.txt    算出来的哈希值的前六位
hello git

```

Hash 算法

- 是把任意长度的输入通过散列算法变换成固定长度的输出

- MD5 128bit
- SHA1 160bit
- SHA256 256bit
- SHA512 512bit

hello git

↓ SHA1

021d153ca2ec1771bb2b0ca33031f9ef5e6946bd 长度40, 16进制

000000100001110100010101001111001010001
011101100000101110111000110111011001010 长度160, 二进制
110000110010100011001100000011000111111
001111011110101111001101001010001101011
1101

- a) -t: type, 查看文件类型。常见文件类型如下

- commit**: 表示一个提交对象, 它记录了一次代码变更的快照, 包括作者、日期、提交信息以及指向父提交的指针。

```

→ git-demo git:(master) git cat-file -t d0bde7c
commit
→ git-demo git:(master) git cat-file -p d0bde7c
tree 29d3f358addb2b6e16ebfb981716fa75cc562ee7
author Peng Xiao <xiaoquwl@gmail.com> 1590009304 +0200
committer Peng Xiao <xiaoquwl@gmail.com> 1590009304 +0200

1st commit

```

- tree**: 表示一个树对象, 它是一个目录结构的快照, 记录了文件和子目录的内容。

```

→ git-demo git:(master) git cat-file -t 29d3f3
tree
→ git-demo git:(master) git cat-file -p 29d3f3
100644 blob ca5a43e53e012b6fd3b2a8a06ebb6c2ee24a24e5    file1.txt
100644 blob 6c493ff740f9380390d5c9ddef4af18697ac9375    file2.txt

```

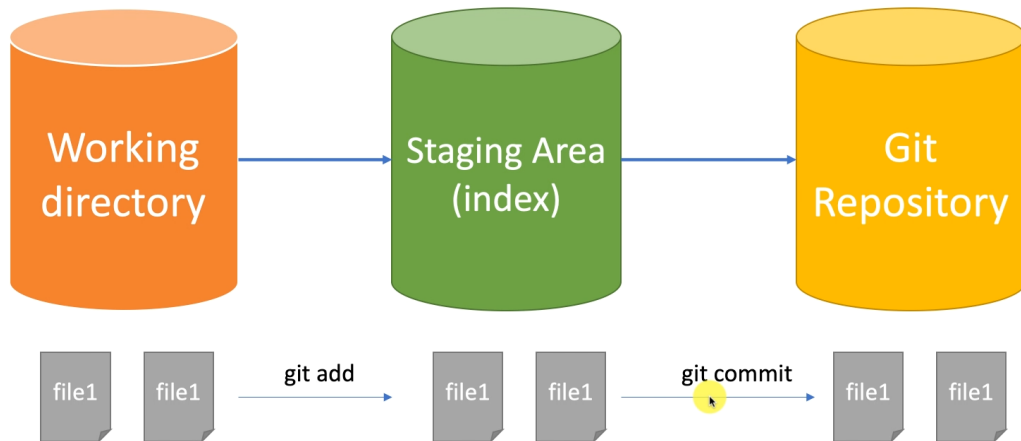
- blob**: 表示一个 blob 对象, 它存储文件的内容。在 Git 中, 每个文件都有一个对应的 blob 对象。**git add** 执行后, 添加文件的类型为 blob。
 - tag**: 表示一个标签对象, 它指向一个特定的提交 (通常是发布版本), 并包含标签名和标签信息。
- b) -p: 根据对象的类型, 以不同的格式输出对象的内容。对于 blob 对象, 它会输出

文件内容；对于 tree 对象，它会输出目录结构等。

```
→ git-demo git:(master) git cat-file -t d0bde7c
commit
→ git-demo git:(master) git cat-file -p d0bde7c
tree 29d3f358addb2b6e16ebfb981716fa75cc562ee7
author Peng Xiao <xiaoquwl@gmail.com> 1590009304 +0200
committer Peng Xiao <xiaoquwl@gmail.com> 1590009304 +0200
1st commit
```

c) -s: size, 查看文件大小。

4. **工作区<-->版本库**：从工作区提交文件到版本库。所有的版本控制系统只能跟踪文本文件的改动，比如在某一行添加了一行代码，或者某一行。修改或添加程序后，.git 目录下的文件并没有改动(因为 git 不保存东西，当前修改仅在工作区中)。通过使用 **git add** 将文件从工作区添加到暂存区，再使用 **git commit** 从暂存区提交到 Git Repository 版本库中。一次 commit 就是一个版本。



```
PS F:\Learning\git\gitTest> git add .\readme.txt
PS F:\Learning\git\gitTest> git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   readme.txt

PS F:\Learning\git\gitTest> git commit -m "change first to 1st, and add a line"
[master 26b8611] change first to 1st, and add a line
 1 file changed, 2 insertions(+), 1 deletion(-)
PS F:\Learning\git\gitTest> git status
On branch master
nothing to commit, working tree clean
```

添加文件到仓库，然后查看仓库状态，确认在主分支中已经提交了修改

提交文件，再查看仓库状态。显示当前工作树是干净的(没有新的内容)

a) 工作区和暂存区

- i. **工作区 working tree**：也就是工作树，也就是 git init 所在的目录，也就是平时存放项目代码的地方。该工作区里的.git 隐藏目录保存了 Git 的跟踪目录。Git 版本库里有很多内容，如下图。

> 此电脑 > Code (F:) > Learning > git > gitTest > .git >

名称	修改日期	类型	大小
hooks	2024/10/8 9:57	文件夹	
info	2024/10/8 9:57	文件夹	
logs	2024/10/8 10:14	文件夹	
objects	2024/10/8 11:11	文件夹	
refs	2024/10/8 9:57	文件夹	
COMMIT_EDITMSG	2024/10/8 11:11	文件	1 KB
config	2024/10/8 9:57	文件	1 KB
description	2024/10/8 9:57	文件	1 KB
HEAD	2024/10/8 9:57	文件	1 KB
index	2024/10/9 9:48	文件	1 KB
ORIG_HEAD	2024/10/9 9:48	文件	1 KB

- ii. **暂存区 stage**: 也称为 index, git add 命令把所有需要提交的修改内容都放到暂存区里。暂存区用于临时存放你的改动, 事实上它只是一个文件(blob 类型的文件), 保存即将提交到文件列表信息。

b) **从工作区提交文件到版本库**

- i. **git add**: 将文件从工作区添加暂存区。执行后没有任何显示就代表成功添加。git add 的本质是当执行完 git add 后, 会创建一些 objects 对象 blob 类型的文件。

```
PS F:\Learning\git\gitTest> git add .\readme.txt
```

./hello.txt	./hello.txt
./.git	./.git
./.git/objects	./.git/objects
./.git/objects/info	./.git/objects/info
./.git/objects/pack	./.git/objects/ce
./.git/hooks	./.git/objects/ce/013625030ba8dba906f756967f9e9ca394464a
./.git/info	./.git/objects/pack
./.git/info/exclude	./.git/hooks
./.git/HEAD	./.git/info
./.git/branches	./.git/info/exclude
./.git/description	./.git/HEAD
./.git/refs	./.git/index
./.git/refs/heads	./.git/branches
./.git/refs/tags	./.git/description
./.git/config	./.git/refs
	./.git/refs/heads
	20 ./git/refs/tags
	21 ./git/config

git add后新增内容

- ii. **git commit**: 将暂存区的文件提交到版本库。使用 -m 对本次提交附带说明(-m 后的内容)。命令执行完后, 会显示本次提交的改动有那些。执行完命令后, 会在 objects 文件夹下添加 commit 类型、tree 类型的文件。每一次 commit, 都会重新生成一个 commit 和 tree 文件。

```
PS F:\Learning\git\gitTest> git commit -m "start our project"
[master (root-commit) cfa1fdb] start our project
1 file changed, 2 insertions(+)
create mode 100644 readme.txt
```

```

→ git-demo git:(master) x git commit -m "1st commit"
[master (root-commit) d0bde7c] 1st commit 执行的git commit命令
2 files changed, 2 insertions(+)
create mode 100644 file1.txt
create mode 100644 file2.txt
→ git-demo git:(master) git cat-file -t d0bde7c 生成的commit类型的文件
commit
→ git-demo git:(master) git cat-file -p d0bde7c
tree 29d3f358addb2b6e16ebfb981716fa75cc562ee7
author Peng Xiao <xiaoquwl@gmail.com> 1590009304 +0200 添加的用户
committer Peng Xiao <xiaoquwl@gmail.com> 1590009304 +0200
1st commit
→ git-demo git:(master) git cat-file -t 29d3f3
tree
→ git-demo git:(master) git cat-file -p 29d3f3
100644 blob ca5a43e53e012b6fd3b2a8a06ebb6c2ee24a24e5 file1.txt
100644 blob 6c493ff740f9380390d5c9ddef4af18697ac9375 file2.txt
→ git-demo git:(master)

```

```

.git
.git/config
.git/objects 生成的commit类型的文件
.git/objects/d0
.git/objects/d0/bde7cdf111e2033c949b0e8aaa8c6b01169277
.git/objects/e2 垃圾文件
.git/objects/e2/129701f1a4d54dc44f03c93bca0a2aec7c5449
.git/objects/ca git add后添加到暂存区的blob类型的文件
.git/objects/ca/5a43e53e012b6fd3b2a8a06ebb6c2ee24a24e5
.git/objects/pack 生成的tree类型的文件
.git/objects/29
.git/objects/29/d3f358addb2b6e16ebfb981716fa75cc562ee7
.git/objects/info
.git/objects/6c git add后添加到暂存区的blob类型的文件
.git/objects/6c/493ff740f9380390d5c9ddef4af18697ac9375
.git/HEAD
.git/info

```

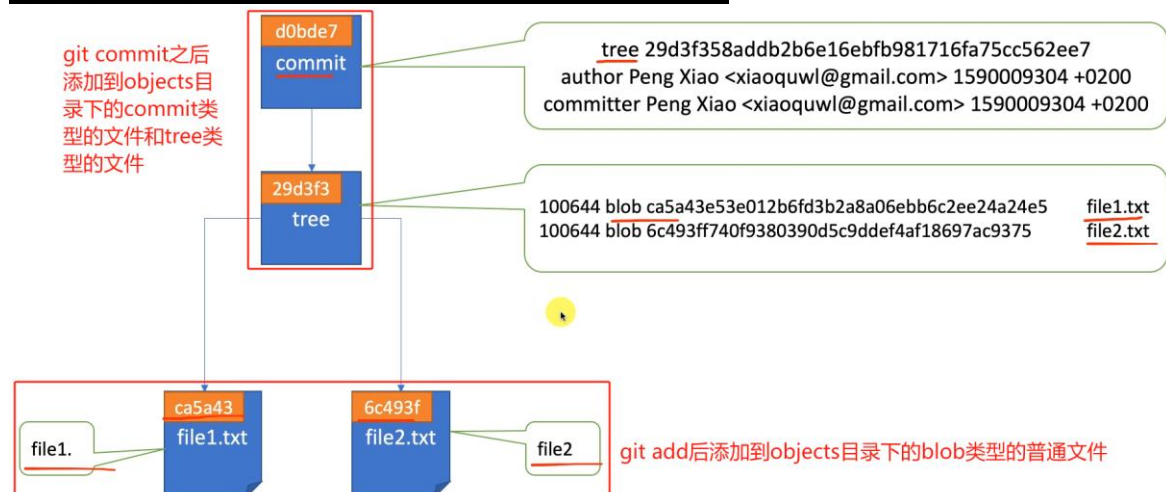


commit 之后，HEAD 指向刚刚提交的版本，也就是新创建的 commit 对象。

```

→ git-demo git:(master) cat .git/refs/heads/master
d0bde7cdf111e2033c949b0e8aaa8c6b01169277
→ git-demo git:(master)
→ git-demo git:(master) cat .git/HEAD
ref: refs/heads/master

```

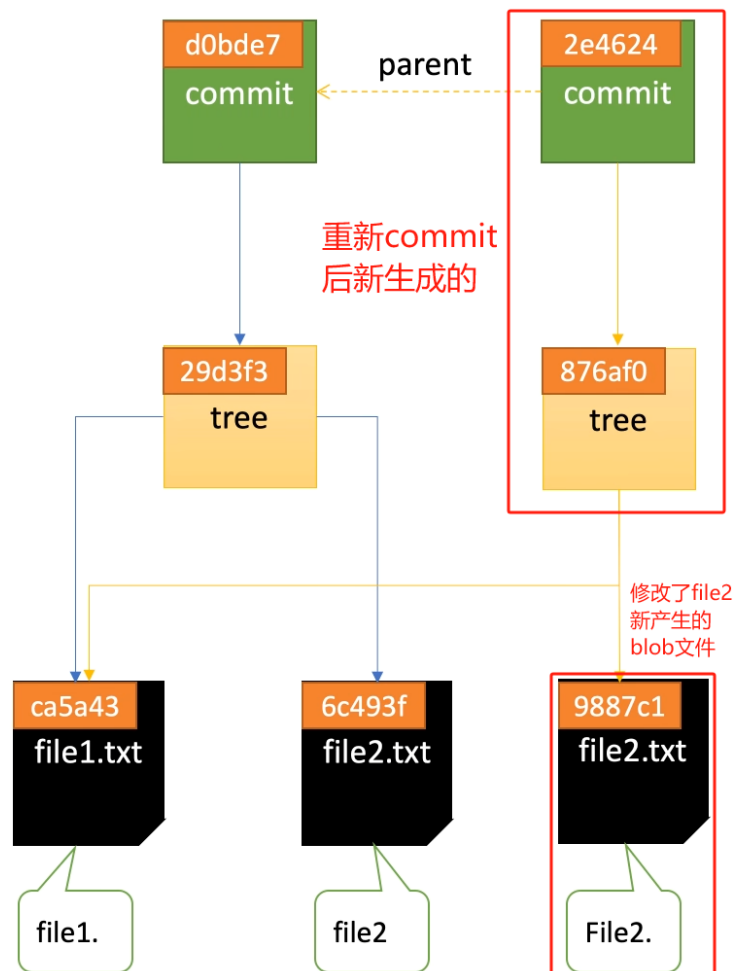


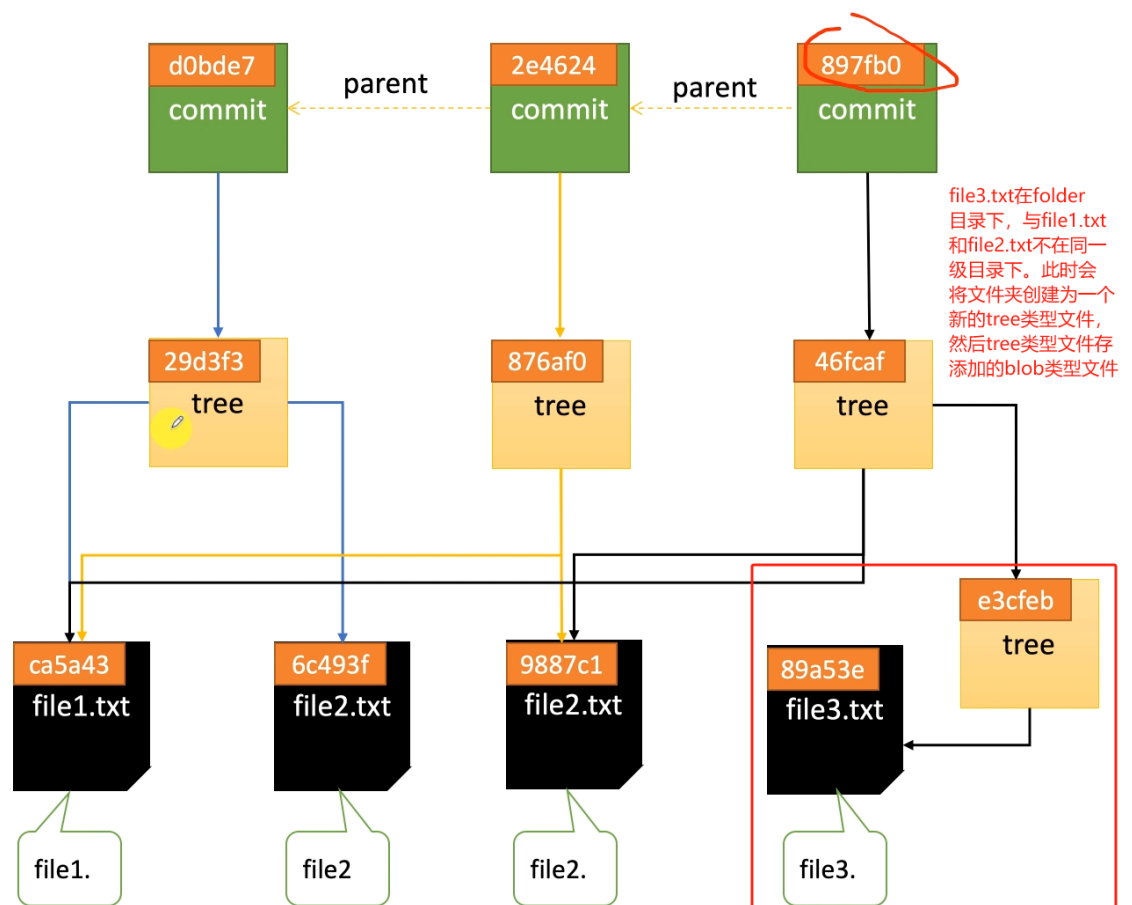
多次 commit 的本质

```
→ git-demo git:(master) * git commit -m "2nd commit"
[master 2e4624f] 2nd commit 修改原有文件, 然后重新add、commit
1 file changed, 1 insertion(+), 1 deletion(-) 新生成的commit
→ git-demo git:(master) git cat-file -p 2e4624f 类型的文件
tree 876af0020e37f45ec09c802d96d7120b2302075f parent: 原来的commit
parent d0bde7cdf11e2033c949b0e8aaa8c6b01169277 类型的文件
author Peng Xiao <xiaoquwl@gmail.com> 1590093358 +0200
committer Peng Xiao <xiaoquwl@gmail.com> 1590093358 +0200

2nd commit
→ git-demo git:(master)
→ git-demo git:(master) git cat-file -p d0bde7cd 原commit类型
tree 29d3f358addb2b6e16ebfb981716fa75cc562ee7 文件
author Peng Xiao <xiaoquwl@gmail.com> 159009304 +0200
committer Peng Xiao <xiaoquwl@gmail.com> 159009304 +0200

1st commit 新生成的tree类型的文件
→ git-demo git:(master) git cat-file -p 876af
100644 blob ca5a43e53e012b6fd3b2a8a06ebb6c2ee24a24e5 file1.txt
100644 blob 9887c1a1fdc70e53d47d73b5764a65c889704ef3 file2.txt
```





- c) 查看工作区和暂存区的状态 `git status`: 用于列出当前工作区和暂存区的状态。执行该命令, 会显示哪些文件被修改了、哪些文件被删除了、哪些文件是新的 (尚未被跟踪) 以及哪些文件已经准备好被提交 (已经暂存)。输出由 **分支信息**、**暂存区信息** 和 **工作区信息** 三个部分组成。

```

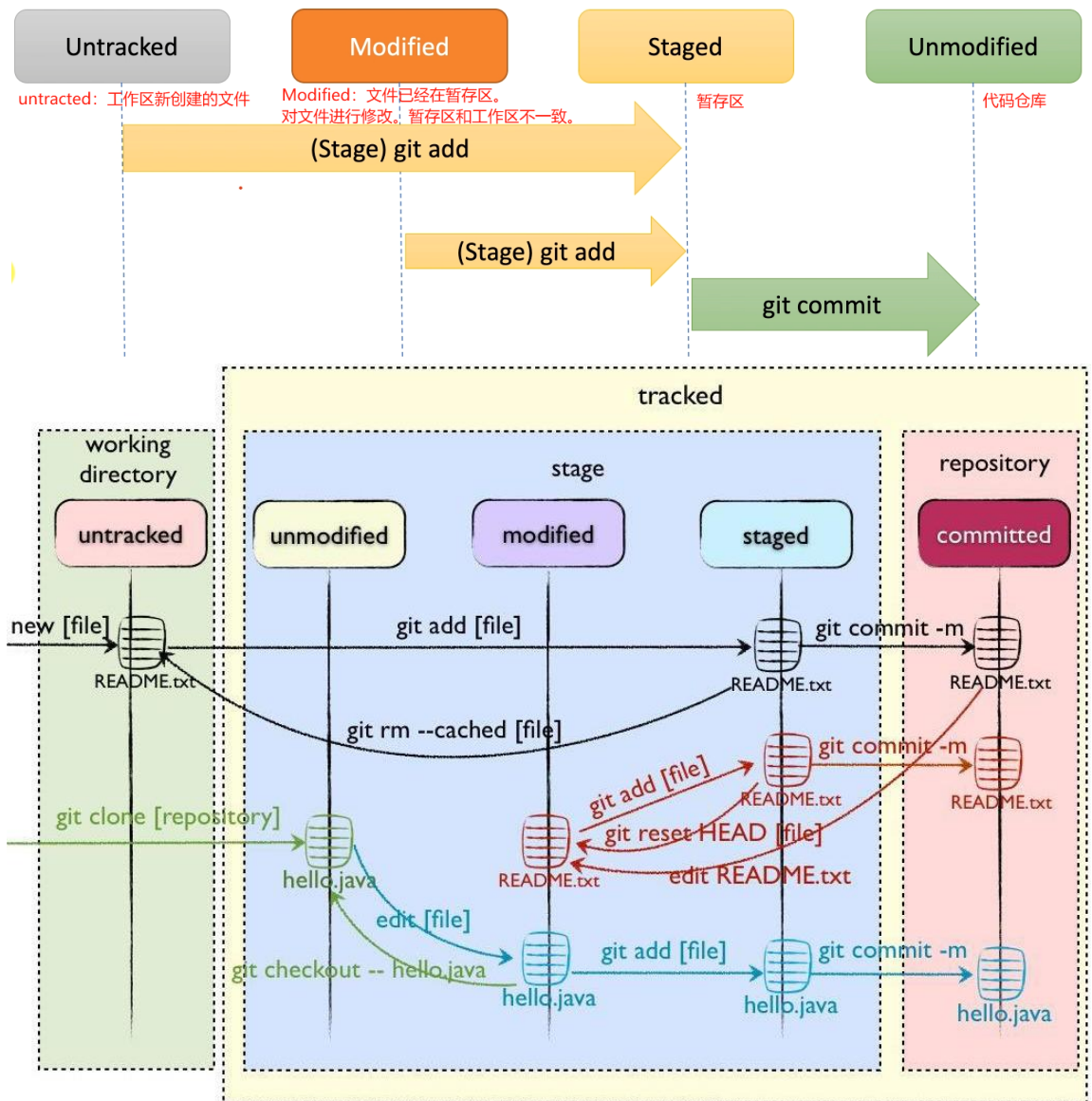
1 On branch main 当前分支为main
2 Your branch is up to date with 'origin/main'.
3
4 Changes not staged for commit: 提示信息, 表示这些更改尚未暂存, 无法修改
5   (use "git add <file>..." to update what will be committed)
6   (use "git restore <file>..." to discard changes in working directory)
7   modified:   file1.txt
8   deleted:    file2.txt
9
10 Untracked files: 提示信息, 表示这些文件没有纳入版本控制
11   (use "git add <file>..." to include in what will be committed)
12   newfile.txt
13
14 no changes added to commit (use "git add" and/or "git commit -a")

```

分支信息: 显示当前所在分支以及分支的状态

暂存区信息: 显示已修改但尚未暂存的文件

工作区信息: 显示了未跟踪的文件。



Untracked: 未跟踪, 此文件在文件夹中, 但并没有加入到git库, 不参与版本控制。通过git add 状态变为Staged。

Unmodify: 文件已经入库, 未修改, 即版本库中的文件快照内容与文件夹中完全一致。这种类型的文件有两种去处, 如果它被修改, 而变为Modified。
如果使用git rm移出版本库, 则成为Untracked文件

Modified: 文件已修改, 仅仅是修改, 并没有进行其他的操作。这个文件也有两个去处, 通过git add可进入暂存staged状态, 使用git checkout 则丢弃修改过, 返回到unmodify状态, 这个git checkout即从库中取出文件, 覆盖当前修改

Staged: 暂存状态。执行git commit则将修改同步到库中, 这时库中的文件和本地文件又变为一致, 文件为Unmodify状态。执行git reset HEAD filename取消暂存, 文件状态为Modified

- d) **git diff**: 查看修改了什么内容, 显示暂存区和工作区文件内容的区别。--代表暂存区, ++代表工作区。

```
PS F:\Learning\git\gitTest> git diff
diff --git a/readme.txt b/readme.txt
index 8041ba8..91ea815 100644
--- a/readme.txt
+++ b/readme.txt
@@ -1,2 +1,3 @@
 Now let us start our project.
-This is the first day.
\ No newline at end of file
+This is the first day.
+I am happy
```

5. 版本回退

- a) **git log**: 查看从最近到最早的 commit 提交日志。-X 参数表示查看最近 X 次提交的版本日志。

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git log 查看从最近到最早的提交日志
commit 8796534424789f6f4bfdd213060ce5fab02cbfd4 (HEAD -> master)
Author: BenXiaoHai <1987884531@qq.com>
Date: Tue Oct 8 11:11:16 2024 +0800
    delete 1st day

commit 1398bd6343d59f323516774473e04bd3377ab981
Author: BenXiaoHai <1987884531@qq.com>
Date: Tue Oct 8 11:10:29 2024 +0800
    add I need food

commit 26b861106732e2efe114ab8b0eaca6a84694c22c
Author: BenXiaoHai <1987884531@qq.com>
Date: Tue Oct 8 10:16:25 2024 +0800
    change first to 1st, and add a line

commit cfa1fdb41439ff873cce3f15a70de8c2a5936ea
Author: BenXiaoHai <1987884531@qq.com>
Date: Tue Oct 8 10:14:27 2024 +0800
    start our project

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git log -1 查看最近一次提交的版本日志
commit 8796534424789f6f4bfdd213060ce5fab02cbfd4 (HEAD -> master)
Author: BenXiaoHai <1987884531@qq.com>
Date: Tue Oct 8 11:11:16 2024 +0800
    delete 1st day
```

commit id 版本号, 是根据用户账号等信息计算出来的一个非常大的数字, 用来避免冲突。

最晚提交

最早提交

- b) **git reset**: 回退到指定的已提交版本。可以通过 HEAD 参数和 commit id 版本号来控制回退到哪个版本。
- i. HEAD 参数: HEAD^表示上一个版本, HEAD^^则表示上个版本。如果想要回退到 10 个版本之前, 则调用 HEAD~10。

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git reset --hard HEAD^
HEAD is now at 1398bd6 add I need food 恢复到上一个提交的版本

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git log
commit 1398bd6343d59f323516774473e04bd3377ab981 (HEAD -> master)
Author: BenXiaoHai <1987884531@qq.com>
Date: Tue Oct 8 11:10:29 2024 +0800
    add I need food

commit 26b861106732e2efe114ab8b0eaca6a84694c22c
Author: BenXiaoHai <1987884531@qq.com>
Date: Tue Oct 8 10:16:25 2024 +0800
    change first to 1st, and add a line

commit cfa1fdb41439ff873cce3f15a70de8c2a5936ea
Author: BenXiaoHai <1987884531@qq.com>
Date: Tue Oct 8 10:14:27 2024 +0800
    start our project
```

git log从三条变成了四条

- ii. commit id 版本号: 使用 commit id 来进行恢复。回退到上一个版本之后, 想恢复到回退之前的版本(还没有关闭 Git Bash 窗口)。

```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git reset --hard HEAD^
HEAD is now at 26b8611 change first to 1st, and add a line

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git log
commit 26b861106732e2efe114ab8b0eaca6a84694c22c (HEAD -> master)
Author: BenXiaoHai <1987884531@qq.com>
Date: Tue Oct 8 10:16:25 2024 +0800

    change first to 1st, and add a line

commit cfa1fdb41439ff873cce3f15a70de8c2a5936ea
Author: BenXiaoHai <1987884531@qq.com>
Date: Tue Oct 8 10:14:27 2024 +0800

    start our project

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git reset --hard 1398bd6 add I need food
HEAD is now at 1398bd6 add I need food

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git log
commit 1398bd6343d59f323516774473e04bd3377ab981 (HEAD -> master)
Author: BenXiaoHai <1987884531@qq.com>
Date: Tue Oct 8 11:10:29 2024 +0800

    add I need food

commit 26b861106732e2efe114ab8b0eaca6a84694c22c
Author: BenXiaoHai <1987884531@qq.com>
Date: Tue Oct 8 10:16:25 2024 +0800

    change first to 1st, and add a line

commit cfa1fdb41439ff873cce3f15a70de8c2a5936ea
Author: BenXiaoHai <1987884531@qq.com>
Date: Tue Oct 8 10:14:27 2024 +0800

    start our project

```

指定回退之间的commit id

git log从2条变成了3条

- c) **git reflog**: 查看之前的全部的 commit 和 reset 命令。可以找到对应的版本 id, 继而恢复到任意版本。

```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git reflog
26b8611 (HEAD -> master) HEAD@{0}: reset: moving to 26b86
26b8611 (HEAD -> master) HEAD@{1}: reset: moving to HEAD^
1398bd6 HEAD@{2}: reset: moving to 1398b
26b8611 (HEAD -> master) HEAD@{3}: reset: moving to HEAD^
1398bd6 HEAD@{4}: reset: moving to HEAD^
8796534 HEAD@{5}: commit: delete 1st day
1398bd6 HEAD@{6}: commit: add I need food
26b8611 (HEAD -> master) HEAD@{7}: commit: change first to 1st, and add a line
cfa1fdb HEAD@{8}: commit (initial): start our project

```

6. **撤销修改**: 修改了文件后, 想要恢复到修改前的样子。有两种情况, 一种是修改前已经 add 并且 commit 过的。另一种是只 add 过但没 commit。**git checkout -** 修改的文件名可以将工作区恢复到最近一次 commit 或者 add 时的状态。
- a) 没有 add, 没有 commit。

```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git status
On branch master
nothing to commit, working tree clean

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ cat readme.txt
Now let us start our project.
This is the first day.
I am happy

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ cat readme.txt
Now let us start our project.
Now I don't need food .
I am happy

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git checkout -- readme.txt

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ cat readme.txt
Now let us start our project.
This is the first day.
I am happy

```

修改文件内容

修改的撤销：没有add，也没有commit

b) 有 add，但没有 commit。

```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ cat readme.txt
Now let us start our project.
Now I don't need food .
I am happy

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git add readme.txt

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ cat readme.txt
Now let us start our project.
Now I don't need food .
I am happy.Haha.

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ cat readme.txt
Now let us start our project.
Now I don't need food .
I am happy.Haha.
Hello.

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git checkout -- readme.txt

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ cat readme.txt
Now let us start our project.
Now I don't need food .
I am happy

```

第一次修改后的文件

add到仓库

第二次修改的文件，但不进行add操作

第三次修改的文件

进行撤销操作，撤销操作恢复到上一次add和commit的文件

已经 add 了某文件，但是想恢复到最后一次 commit 时该文件的状态。

```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ cat readme.txt
Now let us start our project.
Now I don't need food .
I am happy

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git reset HEAD readme.txt
Unstaged changes after reset:
M      readme.txt

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git checkout -- readme.txt

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ cat readme.txt
Now let us start our project.
This is the first day.
I am happy

```

第一句命令首先将暂存区的内容撤销修改（变为 unstage），但不会改变工作区，此时暂存区就变干净了。第二句就是把上次 commit 的内容恢复到工作区。

- c) 有 add，也有 commit。即删除了文件后进行恢复。

如果我们删除了某个被 commit 过的文件，例如 readme.txt，调用 git status，Git 显示你删除了文件。如果你想在 Git 版本库中也删除，则可以调用下面的命令来告诉 Git 你确实想删除该文件，并 commit：

```

$ git rm readme.txt
$ git commit -m "delete readme.txt"

```

如果你只是误删了，则可以使用 checkout 命令来恢复（其实就是使用版本库里面的文件恢复到工作区）：

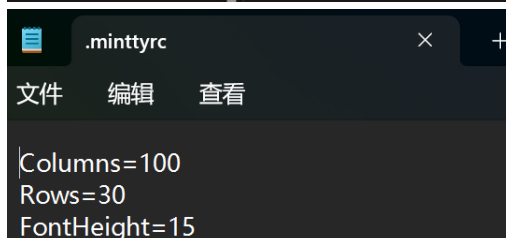
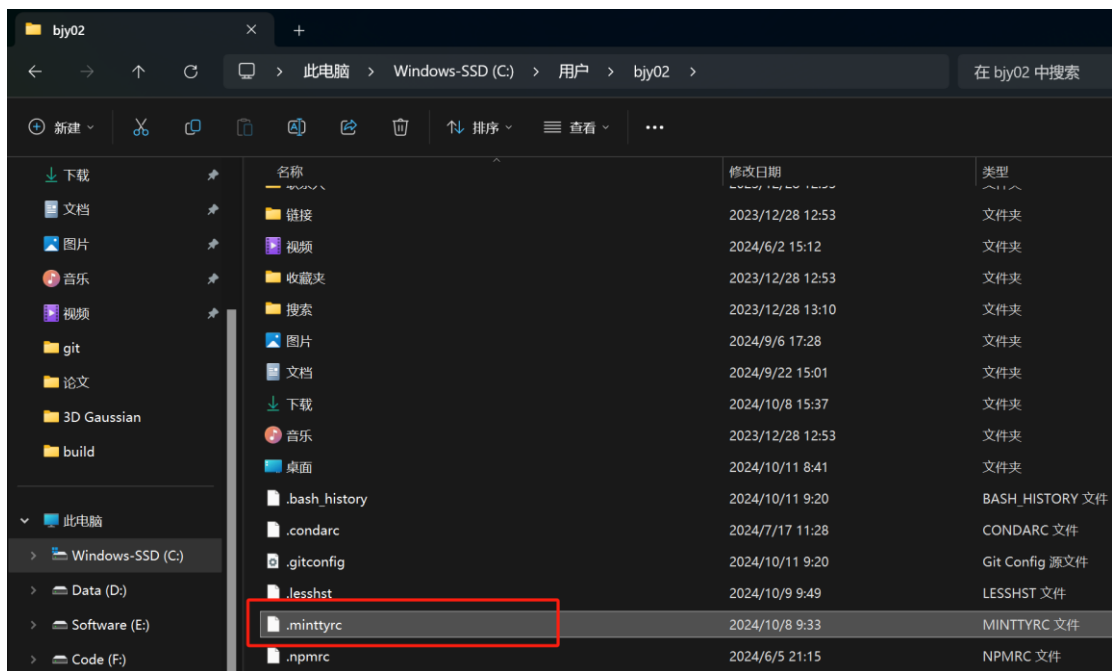
```

$ git checkout -- readme.txt

```

Chapter3 Git 的高阶用法

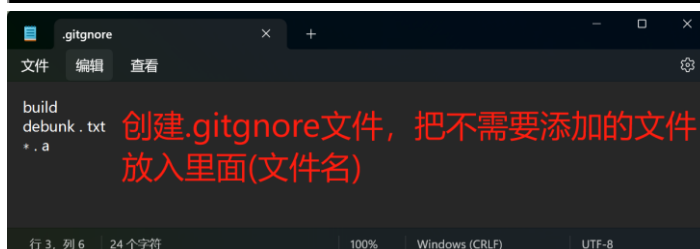
1. **.minttyrc 自定义 Git 文件**：用来自定义 git bash，如设置字体、字体颜色、背景颜色等操作。默认在 C/user/<user names> 文件夹下有一个 **.minttyrc 文件**，用来设置 git 的自定义配置。如果没有可以自己创建一个。



2. **.gitignore 忽略文件**: .gitignore 文件是一个纯文本文件，**指定某些文件和文件夹是 Git 忽略和不追踪的**。文件项目很庞大，不需要被追踪的文件有很多。每次修改查看状态时都会出现海量的 Untracked files 信息(还没有 git add，在工作区创建的文件)。通过创建.gitignore 文件，然后把想要忽略追踪的文件名填写进去，Git 就会自动忽略这些文件。

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  debunk.txt 不重要的文件

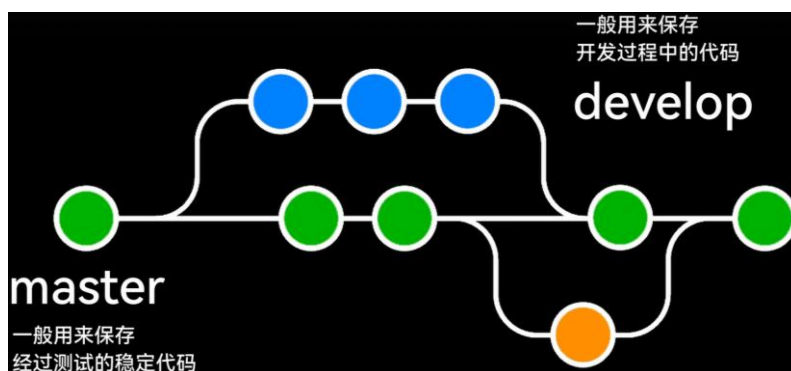
nothing added to commit but untracked files present (use "git add" to track)
```



```
nwu@DESKTOP-GNNPNBD MINGW64 /e/Learning/git/gitTest (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
```


3. 分支与分支管理

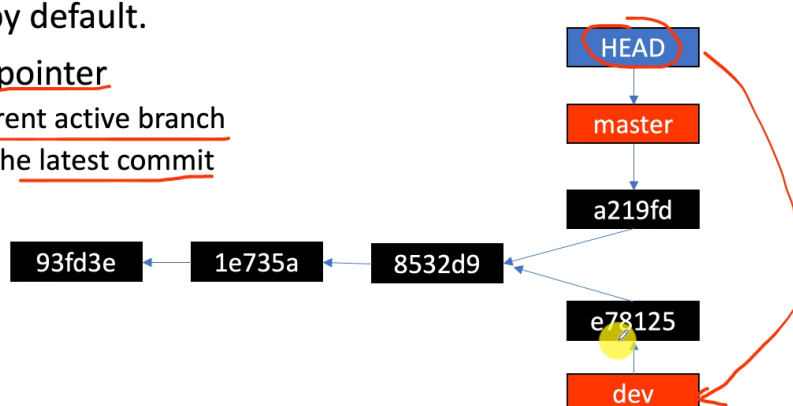


- a) 分支 Branch: Branch are named pointers to commits. 分支是指向各自对应 commit(commit 对象)的指针。Git 分支允许开发者在独立的环境中进行并行开发。在 Git 中, 分支是指代码的一个独立版本。每个分支都有自己的提交历史, 可以在上面进行开发而不影响其他分支。通常有一个主分支 (master 或 main), 用于存放稳定的代码版本。HEAD 指针始终指向当前活跃的分支, 总是指向最近一次的 commit 提交。

Branches are named pointers to commits



- Master is a branch, the canonical mainline branch by default.
- HEAD is a special pointer
 - Related with current active branch
 - Always point to the latest commit



```
→ git-demo git:(master) cat .git/HEAD
ref: refs/heads/master
→ git-demo git:(master) cat .git/refs/heads/master
897fb065bac775a0f9cb9f247b3576e85ba71196
→ git-demo git:(master) git cat-file -t 897f
commit
```

- b) 查看分支:

```
PS F:\postgraduate\TechnologyStack\Git> git branch
* master
PS F:\postgraduate\TechnologyStack\Git> git branch -r
origin/master
PS F:\postgraduate\TechnologyStack\Git> git branch -a
* master
remotes/origin/master
PS F:\postgraduate\TechnologyStack\Git>
```

- i. **git branch**: 查看当前本地仓库的所有分支。

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git branch
bug-solve
dev
dev1
* master
```

- ii. **git branch -r**: 查看远程仓库的所有分支。

```
origin/HEAD -> origin/master
origin/master
```

- iii. **git branch -a**: 列出 Git 仓库中的所有分支，包括本地分支和远程分支。

```
PS F:\postgraduate\TechnologyStack\Git> git branch -a
* master
remotes/origin/master
```

- c) 创建分支 **git branch XXX**: 创建分支。

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git branch dev1 创建git分支, 名为dev1.但是不会立即切换到这个分支
```

- d) 切换分支 **git checkout XXX**: 切换到 XXX 分支。切换分支就是切换 HEAD 指针的指向。注意刚创建两个分支时，两个分支指向相同的 commit 对象，因为此时还没有在新分支下进行 commit 提交。只有在新分支下添加、提交后，两个分支才会指向不同的 commit 对象。

```
→ git-demo git:(master) cat .git/HEAD
ref: refs/heads/master
→ git-demo git:(master) git checkout dev
Switched to branch 'dev'
→ git-demo git:(dev) git branch
→ git-demo git:(dev) cat .git/HEAD
ref: refs/heads/dev
```

切换分支，就是切换 HEAD 指针的指向

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git branch dev1 创建git分支, 名为dev1.但是不会立即切换到这个分支

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git checkout dev1
Switched to branch 'dev1' 切换到dev1分支. 切换工作目录到 dev1分支
D .gitignore
A debunk.txt
A t1.a
A t2.a

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (dev1)
$
```

- e) 创建分支并进行切换 **git switch -c XXX/ git checkout -b XXX**: 直接创建分支 XXX

并进行切换。

- f) 删除分支 `git branch -D XXX`: 不能删除当前所在的分支。**必须切换到其他分支，再执行删除操作**。-D 表示强制删除。删除分支后，分支对应的 commit 对象、tree 对象、blob 对象并不会删除，仍然保留着。这些对象也称为**垃圾对象**。

```
→ git-demo git:(master) git branch -D dev      删除分支
Deleted branch dev (was ab2fb9f).
→ git-demo git:(master) git cat-file -t ab2fb9f
commit
→ git-demo git:(master) git cat-file -p ab2fb9f
tree b48a27332ca027db625a0601ab1c6957fe4f5cc8    分支指向的commit对象
parent 897fb065bac775a0f9cb9f247b3576e85ba71196  的内容
author Peng Xiao <xiaoquwl@gmail.com> 1590355641 +0200
committer Peng Xiao <xiaoquwl@gmail.com> 1590355641 +0200
1st commit from dev branch                        commit对象所指tree对象
                                                的内容
→ git-demo git:(master) git cat-file -p b48a27332c
100644 blob 38f8e886e1a6d733aa9bfa7282568f83c133ecd6    dev.txt
100644 blob ca5a43e53e012b6fd3b2a8a06ebb6c2ee24a24e5    file1.txt
100644 blob 9887c1a1fdc70e53d47d73b5764a65c889704ef3    file2.txt
040000 tree e3cfefb440e2af91732e71deda277030d7182e9    folder1
```

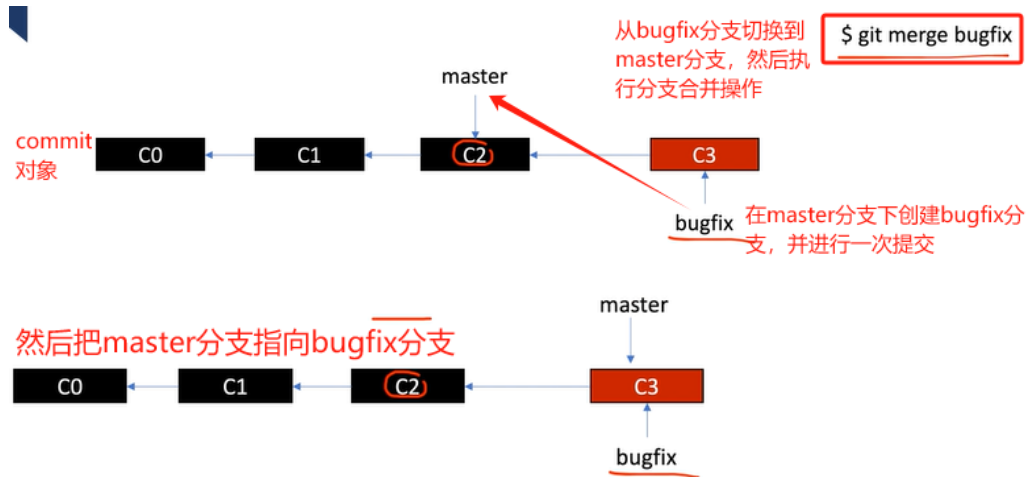
- g) 合并分支 `git merge XXX`: 对新的分支内容修改完了，想要合并到主分支上。**首先使用 `git checkout` 从当前分支切换到 master 主分支，然后使用 `git merge` 合并分支到 master 分支。最后使用 `git branch -D` 删除不需要的分支。就完成了合并操作。默认是 Fast forward 模式。**

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (dev1)
$ git checkout master
Switched to branch 'master' 切换分支

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git merge dev1
Updating 721f39a..fb957ef
Fast-forward
 .gitignore | 3 ---
 debunk.txt | 0
 readme.txt | 2 ++
 t1.a       | 0
 t2.a       | 0
5 files changed, 2 insertions(+), 3 deletions(-)
delete mode 100644 .gitignore
create mode 100644 debunk.txt
create mode 100644 readme.txt
create mode 100644 t1.a
create mode 100644 t2.a    分支的合并。合并dev1分支到master分支

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git branch -d dev1
Deleted branch dev1 (was fb957ef). 删除dev1分支
```

- i. **Fast Forward**: 表示快进模式，**直接把 master 指向新分支所指向的最新的 commit**。Fast Forward 最后的 commit 历史是**一条直线(合并的两个分支是一条直线)**。



具体执行操作如下

```
→ test-only git:(master) git checkout -b bugfix
Switched to a new branch 'bugfix'
→ test-only git:(bugfix)
```

此时因为在 bugfix 分支下还没有进行 add、commit 操作, 所有 bugfix、master 分支指向相同的 commit 对象。

```
commit 8d44826b6df24f5a791a03a28ed6a4d6236c4b72 (HEAD -> bugfix, master)
Author: Peng Xiao <xiaoquwl@gmail.com>
Date: Wed Jun 10 00:13:47 2020 +0200

1st commit
(END)
```

```
commit bd57d7a19e20367096e59a05a30552aade012ef4 (HEAD -> bugfix)
Author: Peng Xiao <xiaoquwl@gmail.com>
Date: Wed Jun 10 00:16:36 2020 +0200

2nd commit
在bugfix分支下进行add、commit, 两个分支指向不同的commit对象

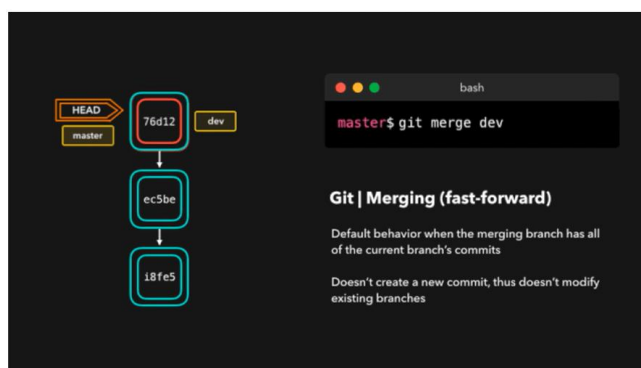
commit 8d44826b6df24f5a791a03a28ed6a4d6236c4b72 (master)
Author: Peng Xiao <xiaoquwl@gmail.com>
Date: Wed Jun 10 00:13:47 2020 +0200

1st commit
(END)
```

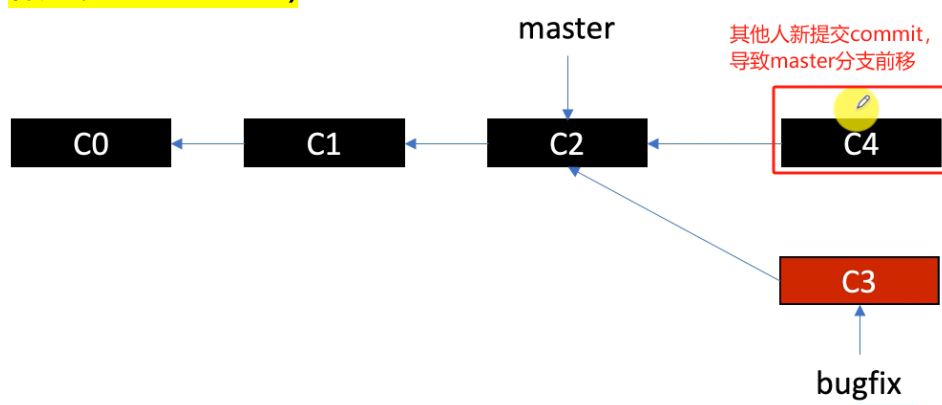
```
→ test-only git:(bugfix) git checkout master
Switched to branch 'master'
→ test-only git:(master) 切换到master分支
```

```
→ test-only git:(master) cat .git/HEAD
ref: refs/heads/master
→ test-only git:(master) cat .git/ORIG_HEAD 新多的文件
8d44826b6df24f5a791a03a28ed6a4d6236c4b72 原来的HEAD指针
→ test-only git:(master) cat .git/refs/heads/master
bd57d7a19e20367096e59a05a30552aade012ef4 现在的HEAD指向的对象
```

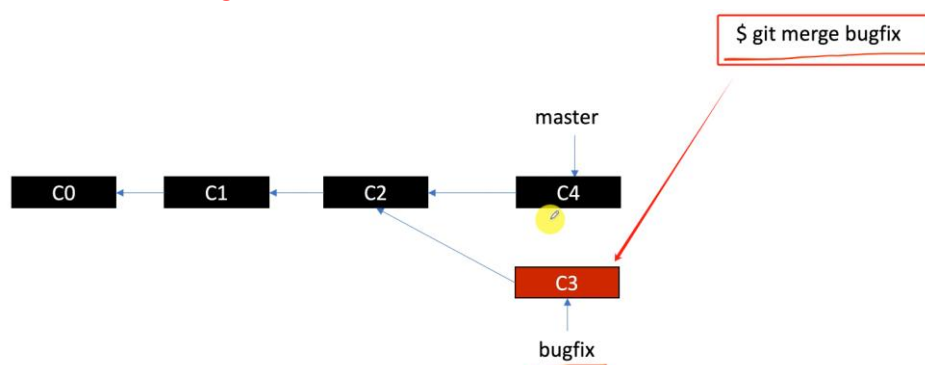
HEAD 指针始终指向当前最近一次提交的 commit 对象。ORIG_HEAD 代表原来的 HEAD 指针, 利用 ORIG_HEAD 可以进行一些回滚操作。



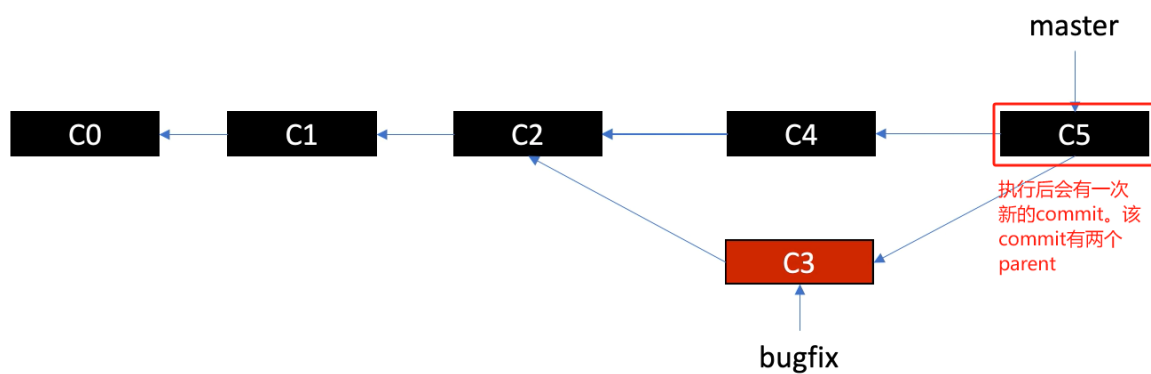
ii. **3 way merge**: 3 way merge 的 commit 历史已具有分叉历史的**树形结构(两个合并的分支成树形结构)**。



master 分支和 bugfix 分支出现分叉的树形结构



git merge 执行完后的情况




```

→ test-only git:(master) x git add test3.txt
→ test-only git:(master) x git commit -m '3rd commit'
[master 148d53f] 3rd commit
1 file changed, 1 insertion(+)
create mode 100644 test3.txt
→ test-only git:(master)

```

此时已有bugfix、master分支在master分支下进行提交，形成3 way merge的情况

```

→ test-only git:(master) git merge bugfix
Merge made by the 'recursive' strategy.
test2.txt | 1 +
1 file changed, 1 insertion(+)
create mode 100644 test2.txt

```

会弹出一个vi界面，需要我们写commit的注释，git自动帮我们commit一次。

```

commit 3b04f8f18859d3c8699ad92630e470a62b5f58be (HEAD -> master)
Merge: 148d53f 02d3ab4
Author: Peng Xiao <xiaoquwl@gmail.com>
Date: Thu Jun 11 00:30:58 2020 +0200

```

自动生成的commit

Merge branch 'bugfix'

```

commit 148d53fa50fd2588a96f3d50700f8549ab2ab4cc
Author: Peng Xiao <xiaoquwl@gmail.com>
Date: Thu Jun 11 00:29:39 2020 +0200

```

原master分支

3rd commit

bugfix分支

```

commit 02d3ab4eeb38d3e6390038c1e9d4ae399fc25178 (bugfix)
Author: Peng Xiao <xiaoquwl@gmail.com>
Date: Thu Jun 11 00:27:29 2020 +0200

```

2nd commit

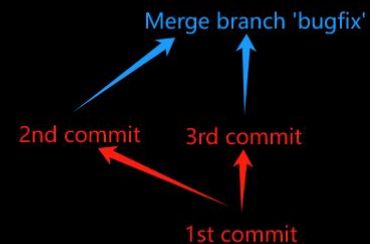
```

commit 48959a520d153b87d6df5ac1a80014d5cba6ea4a
Author: Peng Xiao <xiaoquwl@gmail.com>
Date: Thu Jun 11 00:27:00 2020 +0200

```

1st commit

(END)



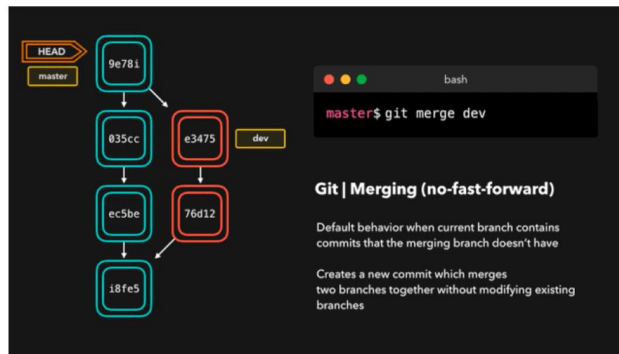
```

→ test-only git:(master) git cat-file -p 3b04f8f18859d3c8
tree c444328b65de4758f86abc7861a2a95cb2ed66b9
parent 148d53fa50fd2588a96f3d50700f8549ab2ab4cc
parent 02d3ab4eeb38d3e6390038c1e9d4ae399fc25178
author Peng Xiao <xiaoquwl@gmail.com> 1591828258 +0200
committer Peng Xiao <xiaoquwl@gmail.com> 1591828258 +0200
Merge branch 'bugfix'

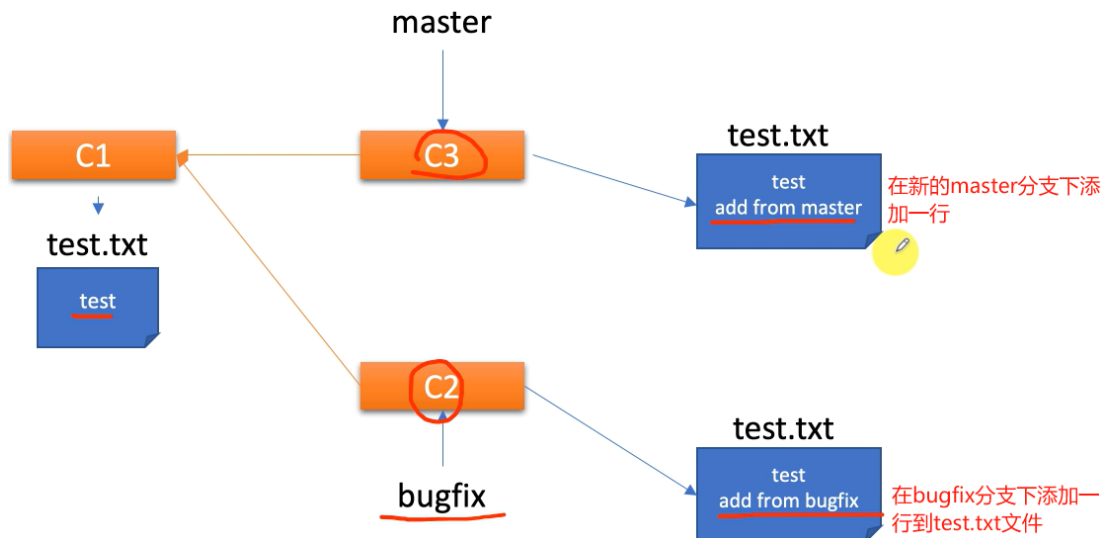
```

有两个parent

Graph	Description	Commit
	master Merge branch 'bugfix'	3b04f8f
	bugfix 2nd commit	02d3ab4
	3rd commit	148d53f
	1st commit	48959a5



- iii. 3 way merge with conflict 冲突处理: 多个分支都对同一个文件进行了修改, 合并时会发生冲突。在 3 way merge 情况下, 在两个分支下均对文件进行了修改, 现在需要合并分支, 文件在合并的过程中会出现冲突。



```

→ test-only git:(master) git merge bugfix
Auto-merging test.txt
CONFLICT (content): Merge conflict in test.txt
Automatic merge failed; fix conflicts and then commit the result.
→ test-only git:(master) x
→ test-only git:(master) x
→ test-only git:(master) x git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   test.txt

no changes added to commit (use "git add" and/or "git commit -a")
  
```

文件合并存在冲突

因为两个分支都对test.txt进行了修改

```
→ test-only git:(master) x cat test.txt
test
<<<<<<< HEAD
add from master
=====
add from bugfix
>>>>>>> bugfix
```

HEAD, 指向的也就是master分支修改的内容

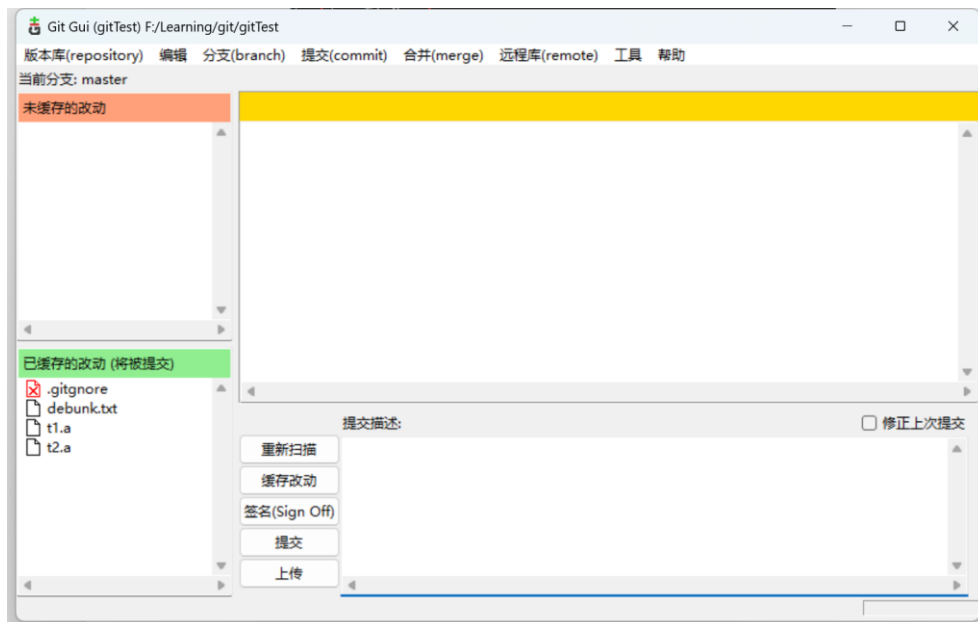
bugfix分支修改的内容

使用vscode打开冲突文件, 修改冲突

```
test.txt
1 test
2 Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
3 <<<<<<< HEAD (Current Change)
4 add from master
5 =====
6 add from bugfix
7 >>>>>>> bugfix (Incoming Change)
```

修改完后重新添加 add、提交 commit。解决冲突。

4. Git 可视化工具：Git GUI 的可视化工具。



- a) 汉化教程: <https://www.cnblogs.com/chenghu/articles/12678500.html>
- b) 基本使用教程: Git GUI 基本操作、分支与合并、远程协作。
<https://segmentfault.com/a/1190000020600546>

Chapter4 远程仓库与多人协作

1. 生成密钥 `ssh-keygen -t rsa -C "1987884531@qq.com"`: 生成 SSH 密钥对的主要目的是为了安全、便捷地进行身份认证。执行该命令后, 生成一个 RSA 类型的 SSH 密钥对, 并且使用 1987884531@qq.com 作为密钥的标签。执行这个命令后, 系统会提示你输入保存密钥对的位置 (默认是在 ~/.ssh/id_rsa), 以及设置一个可选的密码来保护你的私钥。通过 ssh 协议, GitHub 就可以知道当有新的内容被提交上来时, **是你自己的提交的, 而不是别人在冒充你。**

```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ ssh-keygen -t rsa -C "1987884531@qq.com" 生成一个RSA类型的SSH密钥对，并且使用
Generating public/private rsa key pair. 1987884531@qq.com作为密钥的标签。
Enter file in which to save the key (/c/Users/bjy02/.ssh/id_rsa): 生成路径
Created directory '/c/Users/bjy02/.ssh'.
Enter passphrase (empty for no passphrase): 设置密码
Enter same passphrase again: 重复密码
Your identification has been saved in /c/Users/bjy02/.ssh/id_rsa
Your public key has been saved in /c/Users/bjy02/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:CptbHU1uBStYsutc5EZjbbPBKBQsIVYE2y5mKpJcef8 1987884531@qq.com
The key's randomart image is:
+---[RSA 3072]---+
|  ++O+...  |
|  .+...=+O  |
|  ...+B O .  |
|  .. B B =   |
|  +OO.. S =  |
|  .=...*. = O  |
|  =. O =..    |
|  O   O      |
|  .     E     |
+---[SHA256]-----+

```

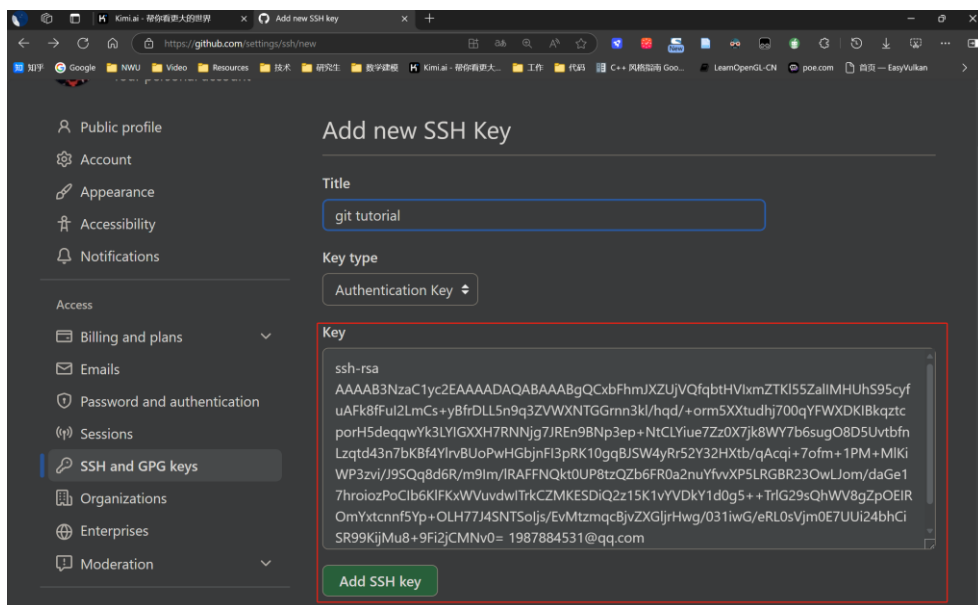
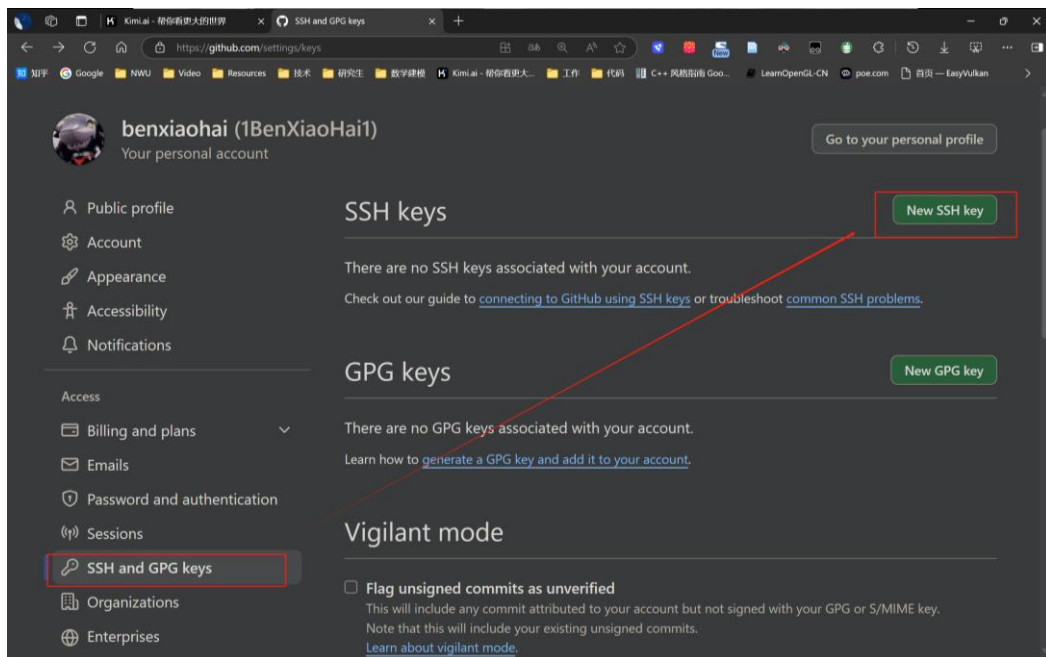


把公钥里面的字符串复制下来，然后在 Github 的 SSH Keys 页面新建一个 SSH Key，把复制的内容粘贴进去。

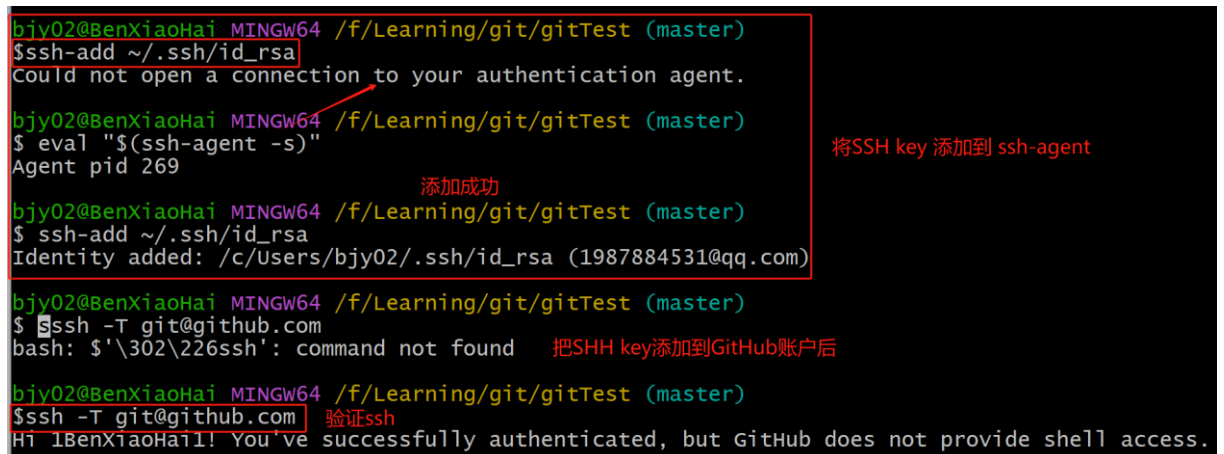
```

ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQGCxbFhmjXZUjVQfqbtHVlXmZTKI55ZallMHUhs95cyfuAFk8fful2LmCs+yBfrDLL5n9q3ZV
WXNTGGGrnn3kl/hqd/
+orm5XXtudhj700qYFWXDKIBkqztcporH5deqqwYk3LYIGXXH7RNNjg7JREn9BNp3ep+NtCLYiue7Zz0X7jk8WY7b6sugO8D5Uvtbf
nLzqtd43n7bKBf4YlrvBUoPwHGbjnF13pRK10gqBJSW4yRr52Y32HXtb/qAcqi+7ofm+
1PM+MIKiWP3zvi/J9SQq8d6R/m9lm/IRAFFNQkt0UP8tzQZb6FR0a2nuYfvvXP5LRGBR23OwLJom/daGe17throiozPoCib6KlFKxWVU
vdwlTrkCZMKESDiQ2z15K1vYVDkY1d0g5
++TrIG29sQhWV8gZpOEIROmYxtcnf5Yp+OLH77J4SNTSoljs/EvMtzmqcBjvZXGljrHwg/031iwG/eRL0sVjm0E7UUi24bhCiSR99Kj
Mu8+9Fi2jCMNv0= 1987884531@qq.com

```



添加完后，需要将 SSH key 添加到 ssh-agent，并且验证一下 key。



2. 远程仓库管理 git remote: git remote 命令是 Git 中用于管理远程仓库的工具。通过

git remote, 可以添加、删除、重命名远程仓库, 并设置或检索它们的 URL。

- a) **git remote**: 列出已经存在的远程分支。

```
→ git-demo git:(master) git remote
origin
```

- b) **git remote -v**: verbose。列出详细信息, 在每一个名字后面列出其远程 url。

```
→ git-demo git:(master) git remote -v
origin https://github.com/git2022/git-demo.git (fetch)
origin https://github.com/git2022/git-demo.git (push)
```

- c) **git remote add origin git@github.com:github 用户名 1BenXiaoHai1/远程仓库名 XXX.git**: 添加一个新的远程仓库, 将本地的 Git 仓库关联到远程仓库。origin 是给远程库取的名字, 在 Git 中属于默认的叫法。之后就可以把本地库的内容推送到远程库上了。

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git remote add origin git@github.com:1BenXiaoHai1/gitTest.git

→ git-demo git:(master) cat .git/config
[core]
    repositoryformatversion = 0
    filemode = true
    bare = false
    logallrefupdates = true
    ignorecase = true
    precomposeunicode = true
[remote "origin"]
    url = https://github.com/git2022/git-demo.git
    fetch = +refs/heads/*:refs/remotes/origin/*
```

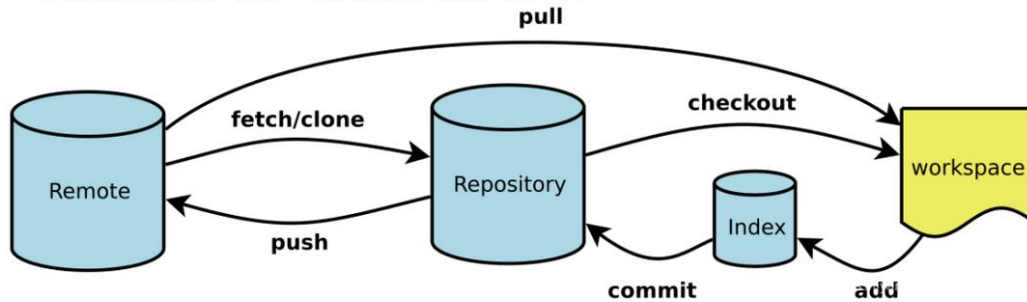
- d) **git remote remove 远程仓库 origin**: 取消关联。

- e) **git remote show 远程仓库 origin**: 显示关于远程仓库的信息。如远程仓库的分支。

```
→ git-demo git:(master) git remote show origin
* remote origin
  Fetch URL: https://github.com/git2022/git-demo.git
  Push URL: https://github.com/git2022/git-demo.git
  HEAD branch: master
  Remote branches:
    master          tracked
    refs/remotes/origin/dev stale (use 'git remote prune' to remove)
  Local branch configured for 'git pull':
    master merges with remote master
  Local ref configured for 'git push':
    master pushes to master (local out of date)
→ git-demo git:(master)
```

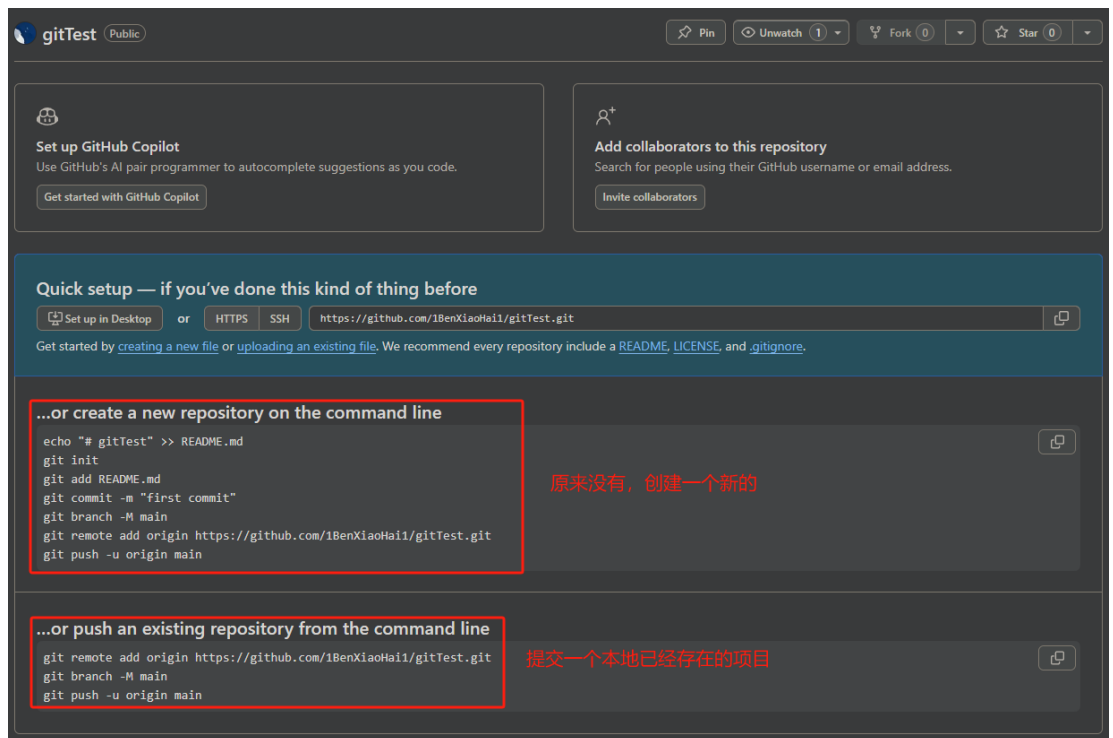
3. **本地仓库<-->远程库的联动**: 把本地的内容推送到远程库上。这样远程库可以作为我们的一个备份, 并且其他人可以通过该仓库来协同工作。也可以把远程仓库克隆到本地, 进行修改, 贡献自己的力量。

Git将项目的存储分为4部分，每部分有自己作用，见下图：



- Workspace：工作区(当前用户操作修改的区域)
 - Index / Stage：暂存区 (add后的区域)
 - Repository：仓库区或本地仓库(commit后的区域)
 - Remote：远程仓库(push后的区域)
- a) **创建一个远程库：**在 Github 网站上点击 [Create a new repository](#)，新建一个 Git 仓库。可以选择一些初始化项。比如仓库的可见性(public、private)、是否添加 README file、忽略文件.gitignore 等。

The screenshot shows the 'Create a new repository' form on GitHub. The form includes the following fields and options:
- **Owner ***: 1BenXiaoHai1
- **Repository name ***: gitTest (with a note 'gitTest is available.')
- **Description (optional)**: git学习的练习仓库
- **Visibility**: Public (selected) and Private (unselected)
- **Initialize this repository with:**
 - ☐ Add a README file
 - ☒ Add .gitignore
 - .gitignore template: None
- **Choose a license**: License: None
- A note at the bottom: 'You are creating a public repository in your personal account.'
- A green button at the bottom right: 'Create repository'



- b) 本地仓库与远程仓库的关联: `git remote add origin git@github.com:github 用户名 1BenXiaoHai1/远程仓库名 XXX.git`。origin 是给远程库取的名字, 在 Git 中属于默认的叫法。之后就可以把本地库的内容推送到远程库上了 (注意此时的用户名和邮箱和远程 GitHub 账号一致)。

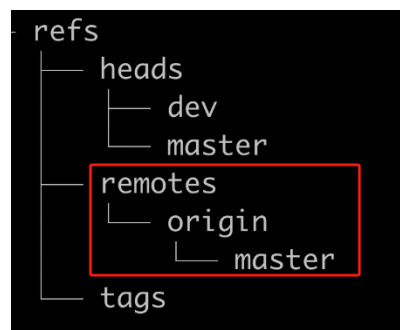
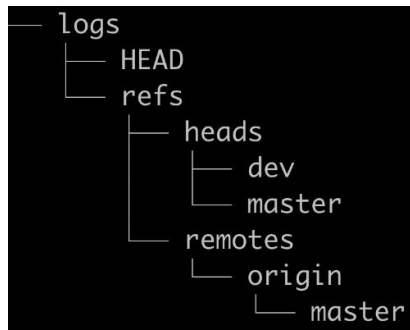
```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git remote add origin git@github.com:1BenXiaoHai1/gitTest.git

→ git-demo git:(master) cat .git/config
[core]
    repositoryformatversion = 0
    filemode = true
    bare = false
    logallrefupdates = true
    ignorecase = true
    precomposeunicode = true
[remote "origin"]
    url = https://github.com/git2022/git-demo.git
    fetch = +refs/heads/*:refs/remotes/origin/*
```

- c) 推送本地仓库到远程仓库: `git push -u origin master`。-u 表示远程 Git 还没有内容, 把本地仓库的 master 分支推送给远程仓库的 master 分支。如果在本地修改了内容, 继续使用 git push 推送本地仓库到远程仓库。

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git push -u origin master
Enumerating objects: 37, done.
Counting objects: 100% (37/37), done.
Delta compression using up to 32 threads
Compressing objects: 100% (28/28), done.
Writing objects: 100% (37/37), 3.09 KiB | 632.00 KiB/s, done.
Total 37 (delta 10), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (10/10), done.
To https://github.com/1BenXiaoHai1/gitTest.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

push 完之后本地.git 目录下文件发生变化，新添加了有关远程仓库的文件。



```
→ git-demo git:(master) cat .git/refs/remotes/origin/master
4b08db03ebdb338e8dfbb5f6144affa8929d0113 本地的master分支和远程的master分支指向同
→ git-demo git:(master) 一个commit对象
→ git-demo git:(master) git cat-file -t 4b08db03ebdb338e8dfbb5f6144affa8929d0113
commit
→ git-demo git:(master) cat .git/refs/heads/master
4b08db03ebdb338e8dfbb5f6144affa8929d0113

commit 4b08db03ebdb338e8dfbb5f6144affa8929d0113 (HEAD -> master, origin/master)
Author: Peng Xiao <xiaoquwl@gmail.com>
Date: Sun May 31 23:59:04 2020 +0200

    add readme
```

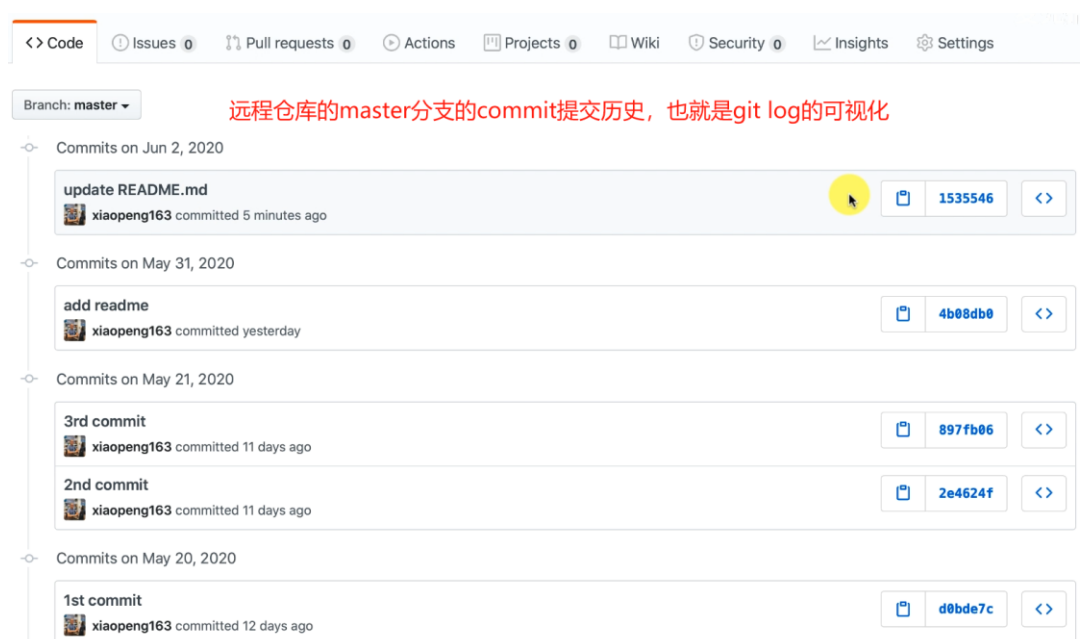
在本地分支修改内容，导致本地分支和远程分支的不同步。通过使用 git push 推送本地分支到远程分支，进行同步。

```
commit 1535546c0f01d5594ef81bd37d7076f2ed552795 (HEAD -> master)
Author: Peng Xiao <xiaoquwl@gmail.com>
Date: Tue Jun 2 00:41:50 2020 +0200

    update README.md

commit 4b08db03ebdb338e8dfbb5f6144affa8929d0113 (origin/master)
Author: Peng Xiao <xiaoquwl@gmail.com>
Date: Sun May 31 23:59:04 2020 +0200

    add readme
```

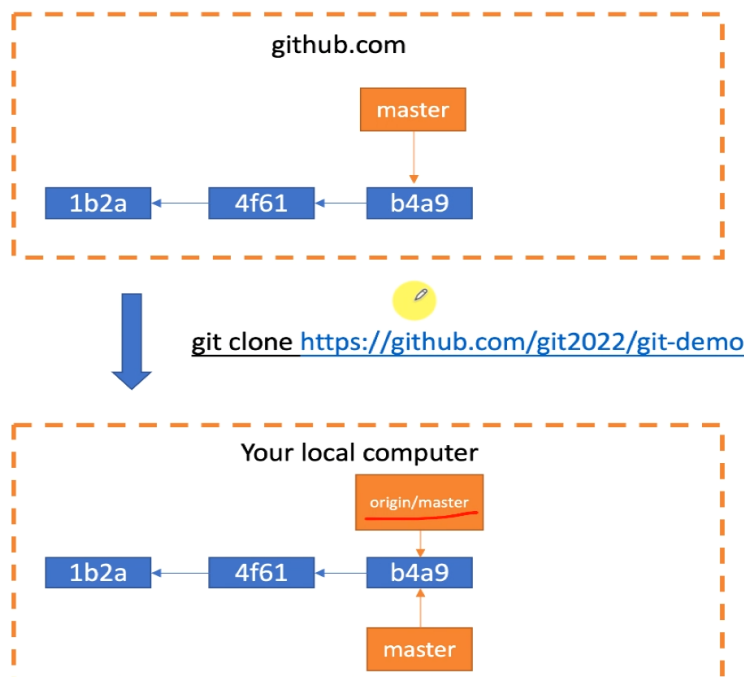


- d) **更新远程仓库到本地仓库**：可以使用克隆 clone 或拉去 pull 方式拉取最新的远程仓库代码。通常情况下，**远程操作的第一步是使用 git clone 从远程主机克隆一个版本库到本地**。本地修改代码后，**每次从本地仓库 push 到远程仓库之前都要先进行 git pull 操作**，保证 push 到远程仓库时没有版本冲突。

- i. **git clone 项目地址 url**：从 0 到 1，将其他仓库克隆到本地，**包括被 clone 仓库的版本变化**。git 文件夹里存放着与远程仓库一模一样的版本库记录。clone 操作是一个**从无到有**的克隆操作，**不需要 git init 初始化**。

```
PS F:\Learning\git\Developer> git clone git@github.com:1BenXiaoHail/gitTest.git
Cloning into 'gitTest'...
remote: Enumerating objects: 37, done.
remote: Counting objects: 100% (37/37), done.
remote: Compressing objects: 100% (18/18), done.
remote: Total 37 (delta 10), reused 37 (delta 10), pack-reused 0 (from 0)
Receiving objects: 100% (37/37), done.
Resolving deltas: 100% (10/10), done.
PS F:\Learning\git\Developer>
```

此电脑 > Code (F:) > Learning > git > Developer > gitTest			在 gitTest 中搜索
名称			修改日期
.git			2024/10/21 14:21
.gitignore			2024/10/21 14:21
debunk.txt			2024/10/21 14:21
readme.txt			2024/10/21 14:21
t1.a			2024/10/21 14:21
t2.a			2024/10/21 14:21
			类型
			文件夹
			Git Ignore 源文件
			文本文档
			文本文档
			A 文件
			A 文件



- ii. **git pull 远程仓库(origin) 远程分支(XXX):本地合并的分支(XXX)**：从 1 到 1'，git pull 作用是，取回远程主机某个分支的更新，再与本地的指定分支 merge 合并。**git pull = git fetch + git merge**。如果远程分支是与当前分支合并，则冒号后面的部分可以省略。可能会产生冲突，需要手动解决。

```
PS F:\postgraduate\Technology Stack\Git> git pull origin main:main
Enter passphrase for key '/c/Users/nwu/.ssh/id_rsa':
Already up to date.
PS F:\postgraduate\Technology Stack\Git> |
```

- iii. **git fetch 远程仓库(origin) 远程分支(XXX):** git fetch 不会进行合并, 执行后需要手动执行 git merge 合并。git fetch 会取回远程的分支, 返回一个 FETCH_HEAD, 指的是某个 branch 在服务器上的最新状态。通过 git log -p FETCH_HEAD 查看刚取回的更新信息。返回的信息包括更新的文件名, 更新的作者和时间, 以及更新的代码 (19 行红色[删除]和绿色[新增]部分)。通过这些信息来判断是否产生冲突, 以确定是否将更新 merge 到当前分支。

```
Windows PowerShell
commit ebbff30a040a0181de020a3f35fef8f87647e10f (HEAD -> main, origin/main)
Merge: a6fe41e 2e2d7a6
Author: BenXiaohai <1987884531@qq.com>
Date: Mon Apr 7 16:08:57 2025 +0800

Merge remote-tracking branch 'remotes/origin/main'

commit a6fe41e28f54d805ce5676680b499e7f92475077
Author: BenXiaohai <1987884531@qq.com>
Date: Mon Apr 7 15:44:47 2025 +0800

First Commit

diff --git a/Developer/gitTest b/Developer/gitTest
new file mode 160000
index 0000000..92051ae
--- /dev/null
+++ b/Developer/gitTest
@@ -0,0 +1 @@
+Subproject commit 92051ae6b49957b43fe8b4450353dd804498aa85
diff --git a/Git学习笔记.docx b/Git学习笔记.docx
new file mode 100644
index 0000000..5381d84
--- /dev/null
+++ b/Git学习笔记.docx
@@ -0,0 +1,253 @@
+
+Chapter1 Git常识
+版本控制: 进行多次开发, 会有多个版本文件。可以将每一次的文件保存下来, 但这种方法存在缺点。因此需要进行版本控制。
+版本控制软件: 在编写程序时, 往往需要多次开发, 并且保留一些历史版本, 以及对不同的需求扩展到新的版本, 版本控制软件可以自动记录一些文件的增删改查, 并且可以随时回溯到历史版本。版本控制软件分为集中式版本控制和分布式版本控制。
+集中式版本控制: 如SVN和CVS。有一个单一的服务器来管理, 这个服务器保存了所有文件的修订和版本, 开发人员连接到服务器
```

```
# 方法一
$ git fetch origin master #从远程的origin仓库的master分支下载代码到本地的origin master
$ git log -p master... origin/master #比较本地的仓库和远程参考的区别
$ git merge origin/master #把远程下载下来的代码合并到本地仓库, 远程的和本地的合并
```

```
# 方法二
$ git fetch origin master:temp #从远程的origin仓库的master分支下载到本地并新建一个分支temp
$ git diff temp #比较master分支和temp分支的不同
$ git merge temp #合并temp分支到master分支
$ git branch -d temp #删除temp
```

- 4.
5. **多人协作:** 在本地工作后, 我们会把本地的提交推送到远程 Git 上, 推送时需要指定推送的分支。哪些分支需要推送, 哪些不需要是需要考量的。在多人协作时, **master 分支用来存放稳定的版本, dev 分支用来存放一些团队工作和修改的结果**。远程 clone 时, 一般默认只会 clone 远程 Git 的 master 分支, 然后需要自己创建远程 origin 的 dev 分支到本地, 然后修改这个 dev 分支, 并把 dev 分支 push 到远程。**如果有两个人都提交了对同一个文件的修改, 就会造成冲突, 此时我们需要后提交的人把前一个提交的人的 commit 给 pull 到自己本地, 然后解决冲突, 再 push 到远程 Git 上去。**

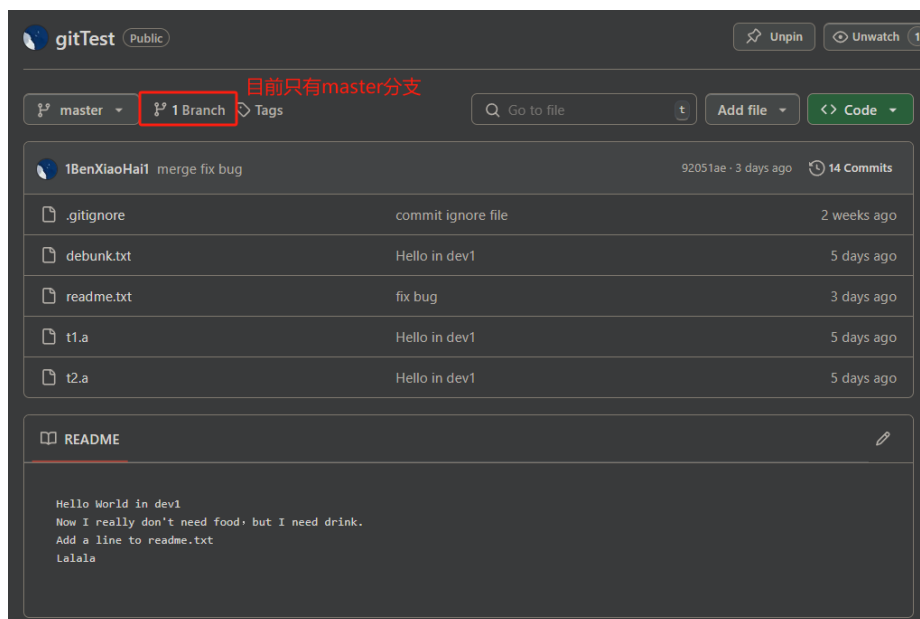
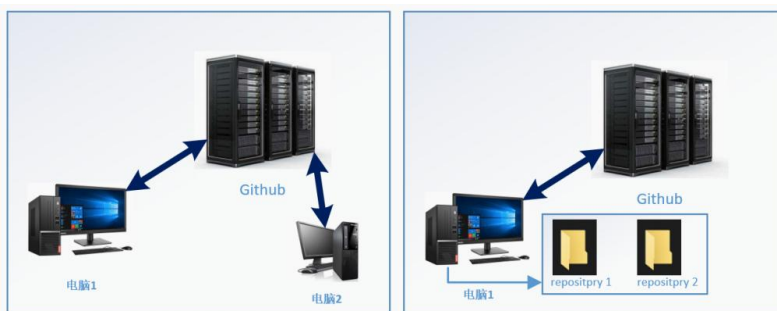
远程仓库所有者

- 创建**远程仓库**（需注册对应托管平台账号）
- 邀请合作者（对方需拥有该托管平台账号）
- 使用Git客户端创建**本地仓库**
- 创建必要的提交（至少一个）
- 为本地仓库配置远程仓库的地址 **HTTPS**
- 将**本地仓库**推送至**远程仓库**

远程仓库合作者

- 接受合作邀请（需注册对应托管平台账号）
- 复制远程仓库地址 **HTTPS**
- 使用Git客户端将**远程仓库**克隆为**本地仓库**

- a) 准备工作：两台电脑与本地建立两个仓库是等价的。目前 Github 的 gitTest 上只有一个 master 分支，与 Github 的 origin 仓库关联，上面有一个 readme.txt 文件。在本地的 Git 仓库创建 dev 分支，修改文件并 commit，然后推送到 Github。



```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git branch -v
  bug-solve 904c4a7 fix bug
  dev1      95fbf26 add some interjections
* master    92051ae merge fix bug

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git switch -c dev
Switched to a new branch 'dev'

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (dev)
$ git add readme.txt 修改后add文件到暂存区

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (dev)
$ git commit -m "add some words"
[dev 69f5537] add some words
1 file changed, 2 insertions(+), 2 deletions(-)

```

```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (dev)
$ git push origin dev
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 32 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 360 bytes | 360.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'dev' on GitHub by visiting:
remote:   https://github.com/1BenXiaoHai1/gitTest/pull/new/dev
remote:
To https://github.com/1BenXiaoHai1/gitTest.git
 * [new branch]      dev -> dev

```

Branches 分支变成了两个，新增加了一个branch New branch

Overview Yours Active Stale All

Q Search branches...

Default

Branch	Updated	Check status	Behind Ahead	Pull request
master	19 hours ago		(Default)	

Your branches

Branch	Updated	Check status	Behind Ahead	Pull request
dev	1 minute ago		0 1	

Active branches

Branch	Updated	Check status	Behind Ahead	Pull request
dev	1 minute ago		0 1	

- b) **Clone 分支**: 创建 myGit1 和 myGit2 两个文件夹。分布在这两个文件夹下克隆 Github 项目。

```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit1
$ git clone git@github.com:1BenXiaoHai1/gitTest.git
Cloning into 'gitTest'...
remote: Enumerating objects: 40, done.
remote: Counting objects: 100% (40/40), done.
remote: Compressing objects: 100% (20/20), done.
remote: Total 40 (delta 11), reused 40 (delta 11), pack-reused 0 (from 0)
Receiving objects: 100% (40/40), done.
Resolving deltas: 100% (11/11), done.

```



```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit2
$ git clone git@github.com:1BenXiaoHai1/gitTest.git
Cloning into 'gitTest'...
remote: Enumerating objects: 40, done.
remote: Counting objects: 100% (40/40), done.
remote: Compressing objects: 100% (20/20), done.
Receiving objects: 100% (40/40), done.
remote: Total 40 (delta 11), reused 40 (delta 11), pack-reused 0 (from 0)
Resolving deltas: 100% (11/11), done.
```

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git
$ cd myGit1

bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit1
$ ls
gitTest/                                     进入myGit1

bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit1
$ cd gitTest/

bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit1/gitTest (master)
$ git branch
* master
```

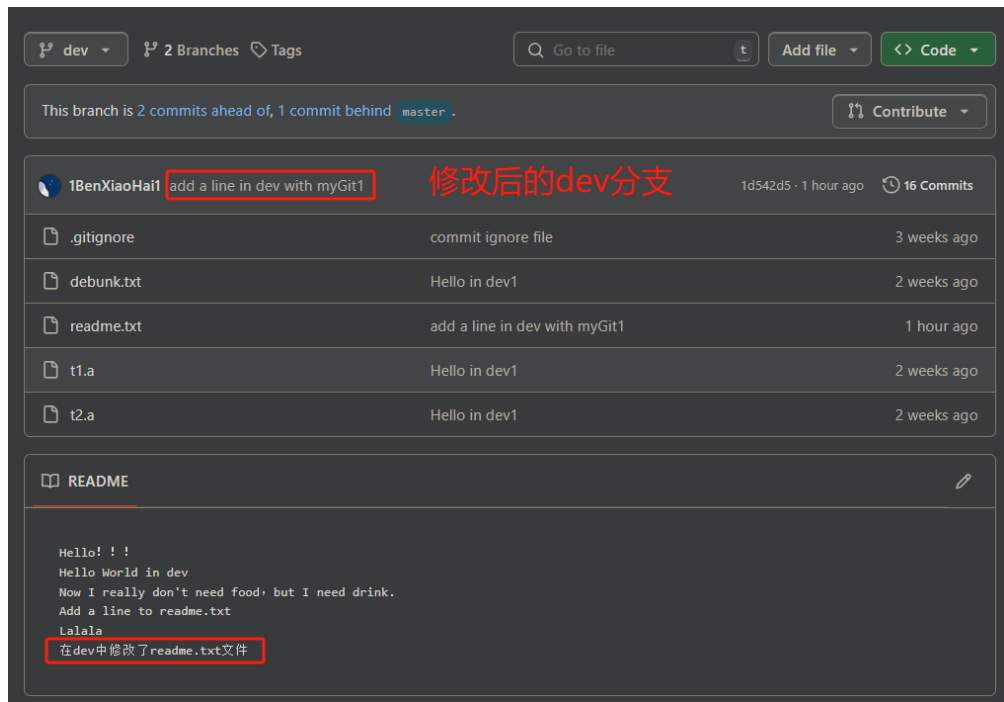
创建本地 dev 分支，并与远程 dev 分支进行连接。

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit1/gitTest (master)
$ git checkout -b dev origin/dev           创建远程origin的dev分支到本地，把本地dev分支和远程dev分支进行连接
Switched to a new branch 'dev'
branch 'dev' set up to track 'origin/dev'.
```

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit1/gitTest (dev)
$ cat readme.txt
Hello! !!
Hello World in dev
Now I really don't need food, but I need drink.
Add a line to readme.txt
Lalala
在dev中修改了readme.txt文件
bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit1/gitTest (dev)
$ git add readme.txt

bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit1/gitTest (dev)
$ git commit -m "add a line in dev with myGit1"
[dev 1d542d5] add a line in dev with myGit1
1 file changed, 1 insertion(+)

bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit1/gitTest (dev)
$ git push origin dev
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 32 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 333 bytes | 333.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:1BenXiaoHai1/gitTest.git
69f5537..1d542d5 dev -> dev           push 到远程服务器上
```



现在本地仓库就是
dev

- c) 多人协作：myGit2 模拟的是另一个团队队员，他在你修改提交 myGit1 的 dev 分支到远程 Git 的 dev 之前就 clone 了 dev。首先在 myGit2 下建立 dev 分支，与 origin/dev 分支建立关联。

意思是它并不知道你的 dev 分支是与远程 Git 的哪个分支相连接的，所以根据提示，我们需要这样操作：

```
git branch --set-upstream-to=origin/dev dev
```

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit2/gitTest (dev)
$ cat readme.txt
Hello! !!
Hello World in dev
Now I really don't need food, but I need drink.
Add a line to readme.txt
Lalala
这是在myGit2的dev分支下的修改!!!
bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit2/gitTest (dev)
$ git add readme.txt
bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit2/gitTest (dev)
$ git commit -m "add a line in dev with myGit2"
[dev 88f6817] add a line in dev with myGit2
1 file changed, 1 insertion(+)
```

修改文件并进行提交

推送到远程，出现冲突。因为远程的 dev 已经被修改，有人新提交了。

冲突解决方法：需要调用 pull 命令来把远程的更新合并到自己本地的工程上，然后再提交。

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit2/gitTest (dev)
$ git push origin dev
To github.com:1BenXiaoHai1/gitTest.git
! [rejected]        dev -> dev (fetch first)
error: failed to push some refs to 'github.com:1BenXiaoHai1/gitTest.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit2/gitTest (dev)
$ git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (1/1), done.
remote: Total 3 (delta 2), reused 3 (delta 2), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 313 bytes | 22.00 KiB/s, done.
From github.com:1BenXiaoHai1/gitTest
69f5537..1d542d5 dev -> origin/dev
Auto-merging readme.txt
CONFLICT (content): Merge conflict in readme.txt
Automatic merge failed; fix conflicts and then commit the result.
```

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit2/gitTest (dev|MERGING)
$ cat readme.txt
Hello!!
Hello World in dev
Now I really don't need food, but I need drink.
Add a line to readme.txt
Lalala
<<<<<<< HEAD
这是在myGit2的dev分支下的修改!!!
=====
在dev中修改了readme.txt文件
>>>>>>> 1d542d5588ce1429ac212dd22dd39d16d9925f4d
```

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit2/gitTest (dev|MERGING)
$ cat readme.txt
Hello!!
Hello World in dev
Now I really don't need food, but I need drink.
Add a line to readme.txt
Lalala
这是在myGit2的dev分支下的修改!!!
在dev中修改了readme.txt文件
```

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit2/gitTest (dev|MERGING)
$ git add readme.txt

bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit2/gitTest (dev|MERGING)
$ git commit -m "solve conflict"
[dev ed5eb22] solve conflict
```

```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit2/gitTest (dev)
$ git push origin dev
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 32 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 695 bytes | 695.00 KiB/s, done.
Total 6 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To github.com:1BenXiaoHai1/gitTest.git
 1d542d5..ed5eb22 dev -> dev

```

推送成功!!!

This branch is 4 commits ahead of, 1 commit behind master.

Commit	Author	Message	Time
ed5eb22	1BenXiaoHai	solve conflict	2 minutes ago
1d542d5	1BenXiaoHai	Hello in dev1	2 weeks ago
...	...	...	...

README

```

Hello!!!
Hello World in dev
Now I really don't need food, but I need drink.
Add a line to readme.txt
Lalala

```

这是在myGit2的dev分支下的修改!!!
在dev中修改了readme.txt文件

push成功

调用 git pull 来将远程 Git 的更新 pull 到本地时显示输出。

```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit2/gitTest (dev)
$ git pull
Already up to date.

```

```

bjy02@BenXiaoHai MINGW64 /f/Learning/git
$ cd myGit1

bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit1
$ cd gitTest/

bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit1/gitTest (dev)
$ git pull
remote: Enumerating objects: 10, done.
remote: Counting objects: 100% (10/10), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 3), reused 6 (delta 3), pack-reused 0 (from 0)
Unpacking objects: 100% (6/6), 675 bytes | 32.00 KiB/s, done.
From github.com:1BenXiaoHai1/gitTest
 1d542d5..ed5eb22 dev -> origin/dev
Updating 1d542d5..ed5eb22
Fast-forward
 readme.txt | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)

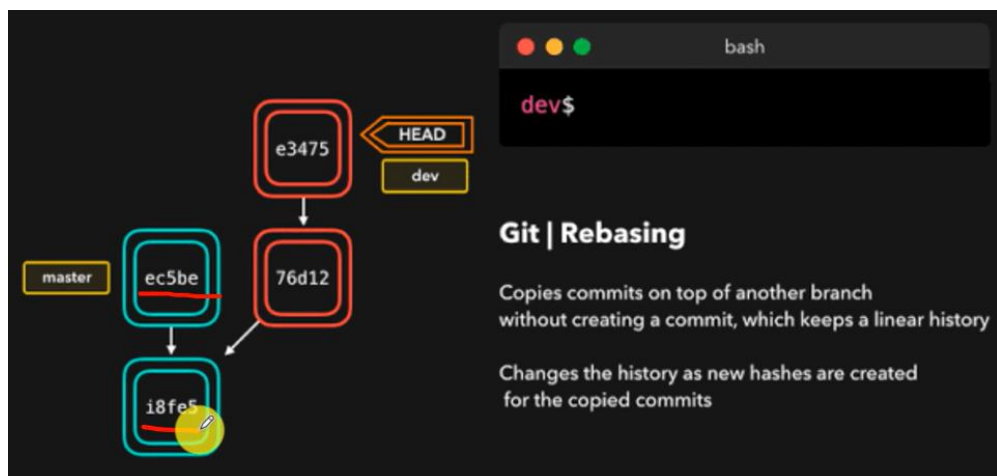
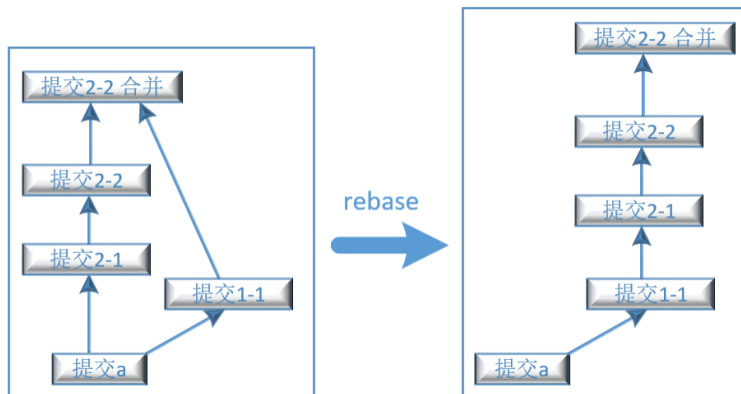
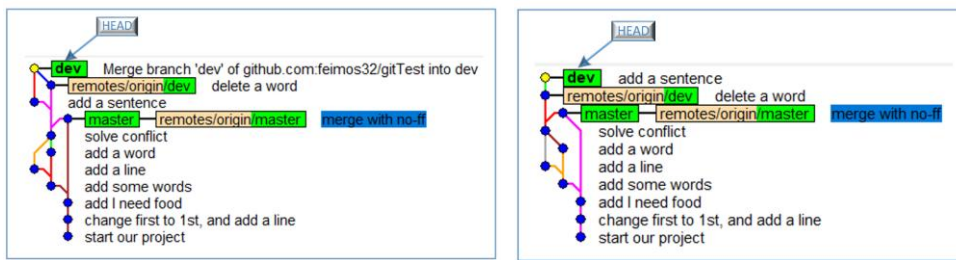
bjy02@BenXiaoHai MINGW64 /f/Learning/git/myGit1/gitTest (dev)
$ cat readme.txt
Hello!!!
Hello World in dev
Now I really don't need food, but I need drink.
Add a line to readme.txt
Lalala
这是在myGit2的dev分支下的修改!!!
在dev中修改了readme.txt文件

```

在myGit1下使用git pull, 将远程的Git更新到本地

d)

6. **git rebase**: 变基。把整个提交历史变为一条干净的线。rebase 可以删除一些历史 commit 等其他功能。



7. 标签管理：想要使用某个版本，可以通过版本的 commit ID 来定位。但是 commit ID 太长，很难记住。因此，通过自定义一些标签来代表特定的版本，本质上是一个指向对应 commit 的指针。想要恢复到某个版本时直接使用标签即可。

- **Create tag** 创建分支

- `git tag <tag name>` create a Lightweight tag (i.e. `git tag v1.0`)
- `git tag -a <tag name> -m <tag message>` create annotated tag that have all the associated meta-data (like email, date, etc.). (i.e. `git tag -a v1.0 -m "version 1.0"`)
- `git tag -a <tag name> <commit SHA1 value>` create annotated tag for an older commit (i.e. `git tag -a v1.0 236e1ef5d755f6ea6894f8f46f36186190be7b92`)

- **List tags** 查看分支

- `git tag`

- **Delete tags:** `git tag -d <tag name>` (i.e. `git tag -d v1.0`) 删除分支

a) 创建标签：会创建两种类型的标签。Lightweight 和 Annotated。



`$ git tag v1.0.0`

`$ git tag -a v1.0.0 -m "version 1.0.0"`

✓ Is stored in the .git/refs/tags

✓ Is stored in the .git/refs/tags

✓ Is also stored in the .git/objects

✓ Tag message

✓ Tag author and date

- i. git tag 标签名：给指定的 commit ID 打标签。如果没有给出 commit ID，则会给当前 HEAD 指针所指向的分支的当前 commit 打上标签。

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git tag v1.0 不给出，默认使用当前HEAD指针所指向的分支

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git show v1.0 查看分支
commit 92051ae6b49957b43fe8b4450353dd804498aa85 (HEAD -> master, tag: v1.0, origin/master)
Merge: 95ed4c0 904c4a7
Author: BenXiaoHai <1987884531@qq.com>
Date: Sat Oct 19 11:23:50 2024 +0800

merge fix bug
```

- ii. git tag -a 标签名 -m 描述标签的信息：创建一个带描述的标签。`-a` 选项表示创建一个 **注解标签 (annotated tag)**，它允许你为标签添加说明。`-m` 选项后面跟着的是你想要为标签添加的说明文字。使用上述命令，会在 objects 对象中创建一个 **tag 对象**。


```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git tag -a v2.0 -m "a tag test" 92051a
给标签做一个说明 (-a 表示版本, -m 表示说明)
bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git show v2.0
tag v2.0
Tagger: 1BenXiaoHai1 <1987884531@qq.com>
Date: Tue Oct 29 10:14:46 2024 +0800
a tag test

commit 92051ae6b49957b43fe8b4450353dd804498aa85 (HEAD -> master, tag: v2.0, tag: v1.0, origin/master)
Merge: 95ed4c0 904c4a7
Author: BenXiaoHai <1987884531@qq.com>
Date: Sat Oct 19 11:23:50 2024 +0800

merge fix bug

```

```

→ git-tags git:(master) git tag -a v1.0.0 -m "version v1.0.0"
→ git-tags git:(master)

```

```

→ git-tags git:(master) git cat-file -t c267
tag
→ git-tags git:(master) git cat-file -p c267
object b8a8bc8c63394642e490b113e6541330e892dec8
type commit
tag v1.0.0
tagger Peng Xiao <xiaoquwl@gmail.com> 1594161583 +0200

version v1.0.0
tag对象的内容

```

- iii. git tag -a 标签名 需要打标签的 commit ID -m 描述标签的信息: 给指定的 commit 对象打标签。

```

→ git-tags git:(master) git tag -a v1.0.0 b8a8 -m "version 1"
→ git-tags git:(master)

```

- b) git show 标签名: 查看指定标签的具体内容。

```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git show v1.0
commit 92051ae6b49957b43fe8b4450353dd804498aa85 (HEAD -> master, tag: v2.0, tag: v1.0, origin/master)
Merge: 95ed4c0 904c4a7
Author: BenXiaoHai <1987884531@qq.com>
Date: Sat Oct 19 11:23:50 2024 +0800

merge fix bug

```

- c) git tag: 查看当前分支的标签。

```

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git tag
v1.0
v2.0

```

- d) 推送标签到远程

- 推送指定标签: git push 远程名 origin 标签名。
- 推送全部标签: git push 远程名 origin -tags。

- e) git tag -d 标签名: 在本地删除标签。

```
bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git tag
v1.0
v2.0

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git tag -d v1.0
Deleted tag 'v1.0' (was 92051ae) 删除标签

bjy02@BenXiaoHai MINGW64 /f/Learning/git/gitTest (master)
$ git tag
v2.0
```

- f) git push origin :refs/tags/v1.0: 删除已经推送到远程的标签。
- 8.